الكلية متعددة التخصصات الناظور
ⵜⴰⵖⵉⵡⴰⵏⵜ ⵜⴰⴳⵜⵉⵣⵍⵉⵜ ⵏ ⵏⵏⴰⵎⵓⵙ
Faculté Pluridisciplinaire de Nador

Université Mohammed Premier
FACULTE PLURIDISCIPLINAIRE DE NADOR

جامعة محمد الأول بوجدة
UNIVERSITÉ MOHAMMED PREMIER OUJDA
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵎⵓⵃⵎⵎⴰⴷ ⴰⵏⴰⵎⵓⵔ ⵓⵊⴷⴰ

Rapport du Projet de Fin d'Études

Filière : Mathématiques Appliquées à l'Intelligence Machine

Approches numériques pour la résolution d'équations différentielles ordinaires

---

Réalisé par :
Abdelouahab KRIBAA
Meryem ESSAKHI

Encadré par :
Pr. Mohamed ARRAI

Soutenu le 23 juillet 2025, Devant le jury :

- Pr. M.ARRAI	Faculté Pluridisciplinaire de Nador (FPN)	(Encadrant)
- Pr. A.Slimani	Faculté Pluridisciplinaire de Nador (FPN)	(Examinateur)

Année universitaire 2024/2025

Remerciements

Au nom d'Allah, le Tout-Miséricordieux, le Très-Miséricordieux.

Nous tenons à exprimer notre gratitude et nos remerciements les plus sincères à **Allah**, notre Seigneur et Créateur, pour nous avoir guidés et nous avoir donné la force et la sagesse pour mener à bien notre projet de fin d'études.

Nous voulons également exprimer notre profonde reconnaissance à notre encadrant, **Mr. ARRAI**, pour ses conseils et son soutien tout au long de ce projet. Votre expertise et votre engagement en faveur de notre succès ont été essentiels pour la réussite de ce travail.

Nous sommes également reconnaissants envers nos professeurs pour leur soutien et leur inspiration tout au long de notre étude. Vos enseignements ont façonné notre vision du monde et ont été essentiels pour nous préparer à ce projet.

Enfin, nous voudrions remercier les **membres du jury** pour leur temps et leur expertise. Votre contribution a été inestimable pour évaluer notre travail et nous aider à nous développer en tant qu'étudiants et futurs professionnels.

Dédicace

Je dédie ce modeste travail à ma chère mère,

À mon père qui me soutient toujours

et aide à affronter les difficultés,

mes sœurs et mes frères,

À tous ceux qui m'aiment

et ceux qui ont contribué directement ou

indirectement à ma réussite, à qui je dédie ce travail,

et leur souhaite une vie remplie de bonheur et de réussite.

Résumé

Ce projet de fin d'études porte sur les équations différentielles ordinaires (EDO), qui sont essentielles pour modéliser des phénomènes dans de nombreux domaines scientifiques et techniques. Le rapport commence par une étude théorique des EDO : définitions, propriétés fondamentales (existence et unicité), méthodes analytiques classiques (séparation des variables, facteur intégrant).

Face aux limitations des méthodes exactes, le travail s'oriente vers les méthodes numériques telles que Euler, Runge-Kutta (ordre 2 et 4) et Heun, permettant d'obtenir des solutions approchées quand une résolution explicite est impossible. Ces méthodes sont appliquées à des cas concrets comme le modèle SIR en biologie, ou la chute libre en physique.

Le rapport présente également une implémentation en Python de ces méthodes, accompagnée de visualisations graphiques pour illustrer la précision des résultats. Ce travail démontre l'importance des méthodes numériques dans l'analyse et la simulation de systèmes dynamiques réels.

Abstract

This final year project focuses on ordinary differential equations (ODEs), which are essential tools for modeling dynamic phenomena in various scientific and engineering fields. The report begins with a theoretical overview of ODEs, including definitions, fundamental properties (existence and uniqueness), and classical analytical methods such as variable separation and integrating factors.

Due to the limitations of exact solutions, the study shifts to numerical methods like Euler, Runge-Kutta (2nd and 4th order), and Heun, which provide approximate solutions when closed-form expressions are unavailable. These methods are applied to practical cases such as the SIR model in biology and free-fall modeling in physics.

The project also includes a Python implementation of the methods, with graphical visualizations to highlight the accuracy of the results. This work underscores the importance of numerical methods in the analysis and simulation of real-world dynamic systems.

Liste des abréviations et notations

EDO : Équation Différentielle Ordinaire

RK : Runge-Kutta

FI : Facteur Intégrant

SIR : Susceptible – Infecté – Retabli (modèle épidémiologique)

LTE : Erreur locale de troncature (Local Truncation Error)

ODE : Ordinary Differential Equation

h **:** Pas de discrétisation

$y(t)$ **:** Solution de l'équation différentielle à l'instant t

y'**,** y'' **:** Dérivée première, dérivée seconde de y

$f(t, y)$ **:** Fonction définissant l'EDO $y' = f(t, y)$

h **:** Pas de discrétisation

t_n **:** Valeur du temps au pas n

y_n **:** Approximation de $y(t_n)$

Table des figures

1.1 Solution de l'équation différentielle $\frac{dy}{dx} = -y$, avec $y(x) = Ce^{-x}$ 12
1.2 Solution de l'équation différentielle $\frac{d^2y}{dx^2} + y = 0$, avec $y(x) = A\cos(x) + B\sin(x)$ 12
1.3 Impact du choix du pas 17

2.1 Méthode d'*Euler* vs. Solution exacte pour EDO : $dy/dx = x + y, y(0) = 1$ 27
2.2 Représentation graphique de la méthode d'*Euler* explicite : approximation par rectangle à gauche. 28
2.3 Représentation graphique de la méthode d'*Euler* implicite : approximation par rectangle à droite. 29
2.4 Comparaison de différentes méthodes Runge-Kutta d'ordre 2 pour $y' = -y$, $y(0) = 1$. 32
2.5 Exemple d'intégration par la méthode de Runge-Kutta-4. 34
2.6 Résolution numérique de y˙(t) + y(t) = 0 avec y(0) = 1 par le schéma classique Runge-Kutta d'ordre 4 34
2.7 methode de heun 36

3.1 Évolution du modèle SIR avec la méthode d'Euler. 43
3.2 Évolution du modèle SIR avec la méthode de Runge-Kutta d'ordre 4. . . 45
3.3 Comparaison entre les résultats des méthodes d'Euler (rouge pointillé) et RK4 (bleu) pour le nombre d'infectés. 46
3.4 La chute libre 47
3.5 I Un corps lâché en chute libre possède une vitesse limite 48
3.6 Résolution de l'équation différentielle par la méthode d'Euler 49
3.7 Interface de simulation du modèle SIR avec grille personnalisable 49
3.8 Simulation spatiale du modèle SIR 50
3.9 Interface principale de simulation 51
3.10 Animation de la chute libre et données en temps réel 52
3.11 Comparaison graphique des méthodes numériques 53

4.1 Comparaison des solutions obtenues par différentes méthodes numériques avec la solution exacte. 62

Table des matières

Remerciements **1**

Dédicace **2**

Résumé **3**

Abstract **4**

Liste des abréviations **5**

Table des figures **6**

1 Théorie des équations différentielles ordinaires **11**
1.1 Introduction 11
1.2 Définition générale d'une équation différentielle ordinaire 11
1.2.1 Ordre, linéarité, forme normale 11
1.2.2 Exemple simple d'EDO 13
1.3 Propriétés fondamentales 13
1.3.1 Existence et unicité (théorème de Cauchy-Lipschitz) 13
1.3.2 Solution exacte 14
1.3.3 Solution approchée 16
1.3.4 Choix du pas 16
1.4 Méthodes analytiques classiques 18
1.4.1 Méthode de séparation des variables 18
1.4.2 Méthode du facteur intégrant (équations linéaires du 1er ordre) . 18
1.4.3 Cas d'équation hétérogène 20
1.5 Conclusion 21

2 Méthodes numériques pour la résolution des EDO **23**
2.1 Introduction 23
2.1.1 Définition et domaines d'application 23
2.2 Discrétisation de l'intervalle 23
2.2.1 Principe de la méthode 23
2.2.2 Formulation mathématique 24
2.2.3 Algorithme pas à pas 25
2.3 Méthode d'Euler 26
2.3.1 Principe de la méthode 26
2.3.2 Formulation mathématique 27
2.3.3 Algorithme pas à pas de la méthode d'*Euler* 29
2.4 Méthode de Runge-Kutta 30
2.4.1 Principe de la méthode 30
2.4.2 Runge-Kutta d'ordre 2 - Algorithme 31
2.4.3 Runge-Kutta d'ordre 4 - Algorithme 33

2.4.4 Erreur de Runge–Kutta 4 34
2.4.5 Exemple Méthode de Heun 35
2.5 Conclusion 38

3 Application dans les different domaine 40
3.1 Introduction 40
3.2 Biologie – Modèle SIR 40
3.2.1 Introduction au modèle SIR 40
3.2.2 Les équations différentielles associées 41
3.2.3 Analyse des graphiques 45
3.3 Physique - Modélisation de la chute libre d'une bille 46
3.3.1 Introduction à la chute libre 46
3.3.2 Méthode numérique de résolution 48
3.4 Simulation graphique du modèle SIR 49
3.4.1 Biologie – Modèle SIR Interface de Simulation 49
3.4.2 Résultats de la Simulation 50
3.4.3 Méthodes numériques utilisées 50
3.5 Simulation graphique de l'application de la chute libre 51
3.5.1 Interface de simulation 51
3.5.2 Animation de la chute et visualisation en temps réel 51
3.5.3 Analyse graphique des méthodes numériques 52
3.6 Conclusion 53

4 Implémentation numérique des méthodes avec Python 56
4.1 Méthodes numériques pour les EDO du premier ordre 56
4.1.1 Méthode d'Euler 56
4.1.2 Méthode de Heun 57
4.2 Transformation des EDO d'ordre supérieur en systèmes 60
4.3 Affichage des solutions graphiques 61

Conclusion Générale 64

Bibliographie 65

Introduction générale

On regroupe sous le terme générique de « méthodes numériques » toutes les techniques de calcul qui permettent de résoudre, de manière exacte ou, le plus souvent, de manière approchée, un problème donné. Le concept de calcul est assez vaste et doit être pris au sens large. Il peut s'agir de déterminer l'inconnue d'une équation, de calculer la valeur d'une fonction en un point ou sur un intervalle, d'intégrer une fonction, d'inverser une matrice, etc.

Bien que la mise en équation d'un problème et sa résolution passent naturellement par les mathématiques, les problématiques sous-jacentes concernent des disciplines aussi variées que la physique, l'astrophysique, la biologie, la médecine, l'économie, etc. Il existe ainsi une grande variété de problèmes possibles, avec pour chacun d'eux des méthodes très spécifiques. De fait, le nombre total de méthodes numériques dont nous disposons à l'heure actuelle est vraisemblablement gigantesque.

L'ordinateur est aujourd'hui un outil incontournable pour simuler et modéliser les systèmes, mais il faut encore savoir exprimer nos problèmes en langage formalisé des mathématiques pures. Nous sommes habitués à résoudre les problèmes de façon analytique, alors que l'ordinateur ne travaille que sur des suites de nombres. On verra dès lors qu'il existe souvent plusieurs approches pour résoudre un même problème, ce qui conduit à des algorithmes différents.

Un algorithme, pour être utile, doit satisfaire un certain nombre de conditions. Il doit être :

— **Rapide** : le nombre d'opérations de calcul pour arriver au résultat escompté doit être aussi réduit que possible.

— **Précis** : l'algorithme doit savoir contenir les effets des erreurs qui sont inhérentes à tout calcul numérique. Ces erreurs peuvent être dues à la modélisation, à la représentation sur ordinateur ou encore à la troncature.

— **Souple** : l'algorithme doit être facilement transposable à des problèmes différents.

,La résolution numérique d'équations différentielles est très souvent nécessaire, faute de l'existence de solutions analytiques.

Chapitre 1 : Théorie des équation différentielles ordinaires

Chapitre 1

Théorie des équations différentielles ordinaires

1.1 Introduction

Les équations différentielles ordinaires (EDO) occupent une place centrale en mathématiques appliquées, en physique et en ingénierie. Elles permettent de modéliser l'évolution dynamique de nombreux phénomènes : le mouvement d'un projectile en mécanique, la croissance d'une population en biologie, ou encore la réponse d'un circuit électrique de type RLC. Les EDO offrent ainsi un cadre rigoureux pour étudier des systèmes dont l'évolution dépend d'une variable, généralement le temps.

Ce chapitre présente les concepts fondamentaux des EDO. Nous commencerons par leur définition générale, suivie de leur classification selon l'ordre, la linéarité et leur écriture sous forme normale. Des exemples concrets illustreront leur usage, accompagnés de théorèmes essentiels tels que les résultats d'existence et d'unicité des solutions. Enfin, nous introduirons des méthodes de résolution analytiques (séparation des variables, facteur intégrant) ainsi que des approches numériques pour les cas où les solutions exactes sont inaccessibles.

L'objectif est de fournir au lecteur les outils théoriques et pratiques nécessaires pour analyser et résoudre une large classe d'équations différentielles ordinaires.

1.2 Définition générale d'une équation différentielle ordinaire

Une équation différentielle ordinaire (EDO) exprime une relation entre une fonction inconnue d'une seule variable indépendante et ses dérivées successives.

Forme générale :

$$f\left(x, y, y', y'', \ldots, y^{(n)}\right) = 0$$

1.2.1 Ordre, linéarité, forme normale

Ordre d'une EDO : L'ordre d'une équation différentielle est défini comme le plus haut ordre de dérivée présent dans l'équation. Voici quelques exemples :

— L'équation $y' + y = 0$ est une EDO **du premier ordre**.

— L'équation $y'' + y = 0$ est une EDO **du second ordre**.

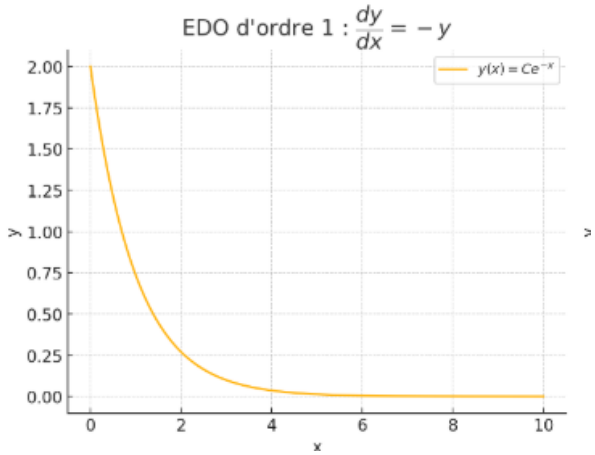

FIGURE 1.1 – Solution de l'équation différentielle $\dfrac{dy}{dx} = -y$, avec $y(x) = Ce^{-x}$

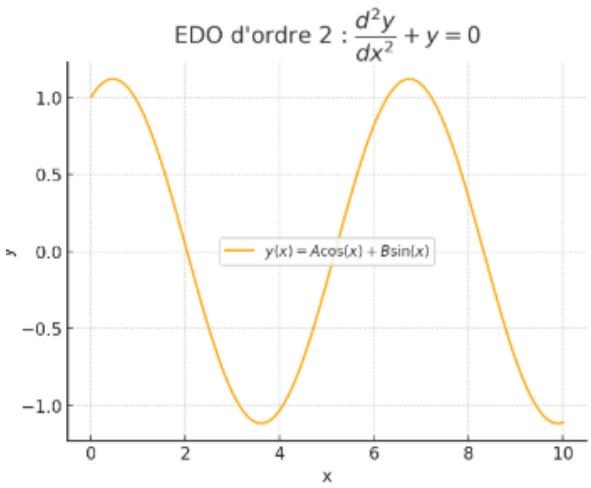

FIGURE 1.2 – Solution de l'équation différentielle $\dfrac{d^2y}{dx^2} + y = 0$, avec $y(x) = A\cos(x) + B\sin(x)$

Linéarité d'une EDO : Une EDO est dite linéaire si la fonction inconnue y et ses dérivées apparaissent de manière linéaire, c'est-à-dire sans être multipliées entre elles ni passer par une fonction non linéaire comme un sinus ou un carré. Sa forme générale est :

$$a_n(x)y^{(n)} + a_{n-1}(x)y^{(n-1)} + \cdots + a_0(x)y = g(x)$$

Elle est dite **homogène** si $g(x) = 0$, sinon elle est **non homogène**.

Exemples :

— Linéaire : $y'' + 3y' + 2y = 0$

— Non linéaire : $y' = y^2, \quad y'' = \sin(y)$

Forme normale : La forme normale d'une EDO consiste à isoler la dérivée d'ordre maximal :

$$y'' = -3y' - 2y$$

Exemple avec second membre :

$$y'' + 3y' + 2y = \sin(x) \quad \Leftrightarrow \quad y'' = \sin(x) - 3y' - 2y$$

Cette forme est essentielle pour la mise en œuvre des méthodes numériques telles que *Euler* ou *Runge-Kutta*, qui s'appuient sur l'approximation de la dérivée la plus élevée.

1.2.2 Exemple simple d'EDO

Considérons l'équation différentielle suivante :

$$\frac{dy}{dt} = -ky$$

Il s'agit d'une EDO linéaire du premier ordre, homogène. Elle peut être résolue par séparation des variables :

$$\frac{dy}{y} = -k\,dt \quad \Rightarrow \quad \ln|y| = -kt + C \quad \Rightarrow \quad y(t) = Ce^{-kt}$$

Interprétation : La solution $y(t) = Ce^{-kt}$ décrit une décroissance exponentielle dans le temps. Ce modèle est typique des systèmes physiques ou techniques où une grandeur diminue proportionnellement à sa valeur actuelle : refroidissement thermique, décharge d'un condensateur, décroissance radioactive, etc.

Remarque : Le paramètre $k > 0$ détermine la vitesse de décroissance. Plus k est grand, plus le système atteint rapidement un état stable (i.e., $y \to 0$ lorsque $t \to \infty$).

1.3 Propriétés fondamentales

1.3.1 Existence et unicité (théorème de Cauchy-Lipschitz)

Dans de nombreux phénomènes physiques, biologiques ou techniques, on modélise le comportement d'un système à l'aide d'une équation différentielle ordinaire (EDO). Considérons le problème de Cauchy suivant :

$$\begin{cases} y'(t) = f(t, y(t)), & t \in [a, b] \\ y(a) = y_0 \end{cases}$$

où $f : D \subset \mathbb{R} \times \mathbb{R} \to \mathbb{R}$, avec D ouvert, et $(ay_0) \in D$.

Définition :

Soit Ω un ouvert de $\mathbb{R}$ et $f : \Omega \to \mathbb{R}$. La fonction f est dite *k-lipschitzienne* s'il existe une constante $k > 0$ telle que :

$$\forall (x, y) \in \Omega^2, \quad \|f(x) - f(y)\| \leq k\|x - y\|.$$

On dit que f est *k-lipschitzienne* .

Théorème de Cauchy–Lipschitz :

Soit $D \subset \mathbb{R} \times \mathbb{R}^n$ un ouvert, et

$$f : D \to \mathbb{R}$$

un champ de vecteurs **continu** et **localement lipschitzien** par rapport à la seconde variable y. Soit un point $(t_0, y_0) \in D$.

Alors, il existe un intervalle ouvert $I \subset \mathbb{R}$ contenant t_0, et une unique fonction

$$y : I \to \mathbb{R}$$

de classe $\mathcal{C}^1$, solution du problème de Cauchy :

$$\begin{cases} y'(t) = f(t, y(t)), & t \in [a, b] \\ y(a) = y_0 \end{cases}$$

sur l'intervalle I.

Démonstration :

Nous cherchons à démontrer l'existence et l'unicité de la solution du problème de Cauchy :

$$\begin{cases} y'(t) = f(t, y(t)), & t \in [a, b] \\ y(a) = y_0 \end{cases}$$

où f est continue et localement lipschitzienne par rapport à la variable y.

Reformulation intégrale :

Une fonction y est solution du problème de Cauchy si et seulement si elle satisfait l'équation intégrale suivante :

$$y(t) = y_0 + \int_a^t f(u, y(u))\, \mathrm{d}u.$$

Cette équation peut être vue comme un point fixe de l'application :

$$\Phi[y](t) = y_0 + \int_a^t f(u, y(u))\, \mathrm{d}u.$$

Contraction de l'application Φ :

On suppose que f est K-lipschitzienne par rapport à y sur un compact D. Pour $y_1, y_2 \in \mathcal{F}$, on a :

$$\|\Phi[y_1](t) - \Phi[y_2](t)\| \leq \int_a^t \|f(u, y_1(u)) - f(u, y_2(u))\|\, \mathrm{d}u \leq K \int_a^t \|y_1(u) - y_2(u)\|\, \mathrm{d}u.$$

En prenant le sup sur t :

$$\|\Phi[y_1] - \Phi[y_2]\|_\infty \leq K\delta \|y_1 - y_2\|_\infty.$$

Si $\delta = b - a > 0$ est choisi tel que $K\delta < 1$, alors Φ est une contraction sur [a,b].

Par le théorème du point fixe de Banach, Φ admet un unique point fixe dans $\mathcal{F}$, qui est la solution du problème de Cauchy. Cela garantit à la fois l'existence et l'unicité de la solution sur un intervalle centré en t_0.

1.3.2 Solution exacte

Le théorème d'existence et d'unicité garantit que, sous certaines conditions de régularité sur la fonction f de l'équation différentielle ordinaire (EDO), il existe une unique solution locale au problème de Cauchy :

$$y' = f(x, y), \quad y(x_0) = y_0.$$

Lorsqu'on parvient à exprimer cette solution de manière analytique, on parle alors de solution exacte ou solution explicite.

Une solution exacte est une expression mathématique explicite, en termes de fonctions usuelles ou spéciales, permettant de calculer directement y à partir de la variable x.

Cas où une solution exacte est possible :

On considère l'équations linéaires du premier ordre de la forme :

$$y' + a(x)y = b(x).$$

On utilise un *facteur intégrant* $\mu(x) = e^{\int a(x)\,dx}$, pour la multiplication.
Exemple :

$$y' - y = e^x \Rightarrow \text{Multiplier par } e^{-x} \Rightarrow \frac{d}{dx}(e^{-x}y) = 1,$$

En intégrant les deux côtés :

$$e^x y = \int 1\,dx = x + C \Rightarrow y(x) = e^{-x}(x + C)$$

• **Équations linéaires à coefficients constants (ordres supérieurs)** : par exemple, une EDO linéaire homogène d'ordre 2 comme :

$$y'' + ay' + by = 0$$

admet pour solution générale une combinaison linéaire d'exponentielles ou de fonctions trigonométriques.
Exemple :

$$y'' + 3y' + 2y = 0$$

a pour équation caractéristique :

$$r^2 + 3r + 2 = 0$$

dont les racines sont -1 et -2, d'où la solution générale :

$$y(x) = C_1 e^{-x} + C_2 e^{-2x}.$$

• **Équations exactes du premier ordre** : si une EDO de la forme :

$$M(x, y)\,dx + N(x, y)\,dy = 0$$

vérifie :

$$\frac{\partial M}{\partial y} = \frac{\partial N}{\partial x},$$

alors elle possède une fonction potentielle $F(x, y)$ telle que $dF = 0$.
On résout $dF = 0$ pour obtenir implicitement $F(x, y) = C$, ce qui fournit la solution exacte de l'EDO.

Pourquoi des méthodes numériques ?

Dans la majorité des cas pratiques, la solution exacte d'une équation différentielle ordinaire (EDO) ne peut pas être obtenue sous forme fermée à l'aide de fonctions élémentaires. Les raisons en sont multiples :

• L'équation peut être fortement non linéaire ou avoir une structure complexe, rendant l'intégration analytique impossible.

• Les coefficients de l'EDO peuvent dépendre de la variable x de manière trop compliquée pour permettre un calcul direct des primitives.

• Les EDO d'ordre élevé, ou les systèmes couplés, ne se résolvent généralement pas par les méthodes élémentaires usuelles.

Face à ces limitations, il est nécessaire d'utiliser des **méthodes numériques d'approximation** pour obtenir une solution approchée. Ces méthodes, que nous introduirons dans le chapitre suivant, permettent de traiter des problèmes plus complexes de manière discrète et efficace.

1.3.3 Solution approchée

Définition :
La **solution approchée** d'une équation différentielle ordinaire (EDO) correspond à une estimation numérique de la solution exacte, obtenue à l'aide d'un algorithme. Partant d'une condition initiale $y(x_0) = y_0$, on calcule une suite de points :

$$(x_0, y_0),\ (x_1, y_1),\ \ldots,\ (x_N, y_N)$$

où chaque valeur y_n est une estimation numérique de la solution en x_n, c'est-à-dire une approximation de $y(x_n)$.

Cette démarche est nécessaire car, dans la majorité des cas, les EDO ne peuvent pas être résolues analytiquement : seules certaines classes d'équations (linéaires, à coefficients constants, séparables, etc.) admettent des solutions exprimables à l'aide de fonctions élémentaires. En revanche, les équations plus complexes comme les EDO non linéaires ou couplées exigent une approche numérique.

La qualité de l'approximation dépend essentiellement du **pas de calcul** h. En règle générale, plus ce pas est petit, plus l'approximation est précise, car l'erreur globale tend vers zéro lorsque $h \to 0$. Il est donc crucial d'utiliser un schéma numérique **convergent** et **stable**, de manière à garantir que la suite (y_n) reste proche de la solution réelle.

L'approche numérique présente l'avantage de permettre un contrôle explicite de la précision : en affinant le maillage, on peut faire tendre la solution approchée vers la solution exacte. Cette capacité suffit généralement pour analyser efficacement le phénomène modélisé, même si elle s'accompagne d'un coût de calcul plus élevé.

1.3.4 Choix du pas

Le choix du pas h est un paramètre central dans l'approximation numérique des EDO. Mathématiquement, le pas h correspond à la distance entre deux points consécutifs du maillage sur l'intervalle de résolution.

Si l'on considère un intervalle $[a, b]$ divisé en n sous-intervalles égaux, alors :

$$h = \frac{b-a}{n}$$

où $h > 0$ est le pas de discrétisation.

Un pas trop grand peut entraîner des erreurs importantes ou rendre le schéma instable, tandis qu'un pas trop petit augmente le coût de calcul. On examine ci-dessous les aspects clés du choix du pas.

Définition : Erreur locale et Erreur globale :
On considère la partition uniforme suivante :

$$[a, b]$$

avec des nœuds de partition $[a, b]$,et $h = \frac{1}{n}$.

L'erreur de consistance (erreur locale) est celle produite au cours d'une itération de la méthode numérique à un pas, à partir de la valeur exacte $Y_n = Z(t_n)$:

$$e_n : Z(t_{n+1}) - Y_{n+1}.$$

L'erreur globale ,e_n représente l'écart maximal, sur l'ensemble des points de discrétisation, entre la solution approchée Y_n et la solution exacte $Z(t_n)$:

$$e_n : \max_{0 \leq n \leq N} |Y_n - Z(t_n)| \to h,$$

avec h le pas de discrétisation ou la norme de la partition.

Le choix du pas h d'intégration fait face à des exigences contradictoires :

• Pour une méthode d'ordre p, plus h est petit, plus l'erreur de consistance locale diminue.

• Pour un domaine d'intégration fixé, une diminution de h augmente le nombre d'itérations, ce qui renforce les erreurs d'arrondi accumulées.

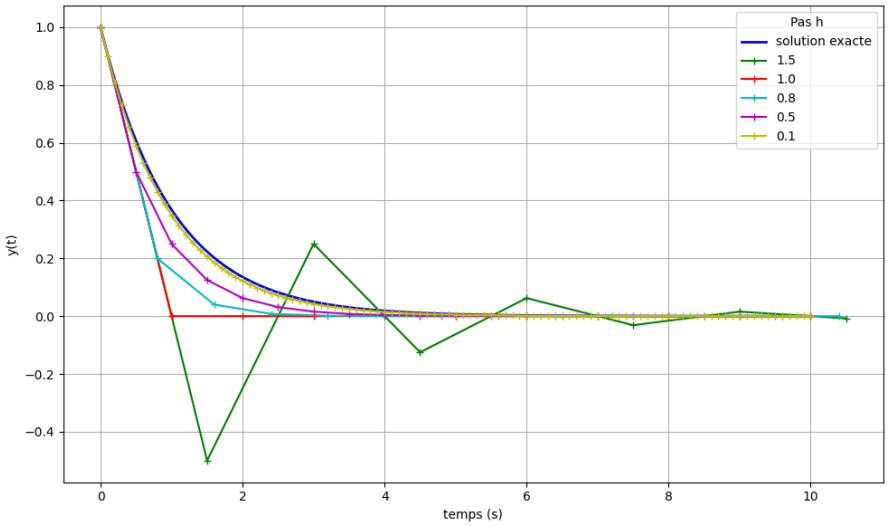

FIGURE 1.3 – Impact du choix du pas

Ce graphe illustre l'impact du choix du pas h sur la précision de la méthode d'Euler (*la méthode d'Euler sera présentée et analysée en détail dans le chapitre suivant*) pour résoudre l'équation différentielle :

$$\dot{y}(t) + y(t) = 0 \quad \text{avec} \quad y(0) = 1$$

Le graphe montre que plus le pas h est petit, plus la méthode d'Euler approche bien la solution exacte $y(t) = e^{-t}$. Un grand pas engendre une erreur importante, tandis qu'un petit pas améliore la précision, au prix de davantage de calculs.

Contrôle du pas :

Il est souvent difficile de déterminer à l'avance la valeur maximale du pas $h_{\max}$ pour garantir que l'erreur globale reste inférieure à une tolérance ε. Les algorithmes à pas variables présentent à cet égard des avantages majeurs :

• Grâce à une estimation « à la volée » de l'erreur de consistance e_n il est possible d'ajuster dynamiquement le pas h_n tout au long de l'intégration.

• Lorsqu'on se rapproche d'une discontinuité ou d'une singularité dans l'équation différentielle, on doit en général réduire très fortement le pas.
Dans cette situation, il est recommandé d'interrompre l'algorithme avant de franchir la discontinuité, sans quoi les erreurs deviennent imprévisibles, car le schéma peut devenir instable ou totalement invalide. Le calcul h_n de fournit alors également un test d'arrêt.

1.4 Méthodes analytiques classiques

1.4.1 Méthode de séparation des variables

Considérons une équation différentielle ordinaire (EDO) de la forme :

$$\frac{dy}{dx} = \frac{f(x)}{g(y)}.$$

Une telle équation est dite *à variables séparables*, parce qu'elle peut être réarrangée pour obtenir seulement y sur le côté gauche et seulement x sur le côté droit.
Ce processus est appelé *ségrégation des variables*. En multipliant par $g(y)$, on obtient :

$$g(y)\frac{dy}{dx} = f(x).$$

C'est le type d'équation que l'on obtient lorsqu'une fonction implicite est différenciée.
Si nous pouvons trouver des fonctions $G(y)$ et $F(x)$ telles que $G'(y) = g(y)$ et $F'(x) = f(x)$, alors l'équation peut s'écrire sous la forme :

$$G'(y)\frac{dy}{dx} = F'(y).$$

Par analyse, le terme à gauche est :

$$\frac{d}{dx}G(y) = F'(y),$$

En intégrant les deux côtés, on obtient :

$$G(x) = F(y) + C.$$

1.4.2 Méthode du facteur intégrant (équations linéaires du 1er ordre)

(EDO) linéaire du premier ordre n'est ni séparable, ni exacte, on peut la rendre exacte en la multipliant par un terme appelé « facteur d'intégration ».

Considérons l'EDO linéaire non homogène du premier ordre ci-dessous, et supposons qu'elle est inexacte :

$$a_0(x)y + a_1(x)\frac{dy}{dx} = f(x).$$

Puisque $a_1(x) \neq 0$, nous pouvons diviser par $a_1(x)$ pour obtenir :

$$\frac{dy}{dx} + \frac{a_0(x)}{a_1(x)}y = \frac{f(x)}{a_1(x)}.$$

Posons alors :

$$P(x) = \frac{a_0(x)}{a_1(x)} \quad \text{et} \quad Q(x) = \frac{f(x)}{a_1(x)}.$$

L'équation devient alors :

$$\frac{dy}{dx} + P(x)y = Q(x).$$

peut être rendue exacte en la multipliant par un facteur d'intégration (FI). Soit le facteur d'intégration$\mu(x)$, et supposons que l'on multiplie

$$\frac{dy}{dx} + P(x)y = Q(x)$$

par $\mu(x)$ fait exact :

$$\mu(x)\frac{dy}{dx} + \mu(x)P(x)y = \mu(x)Q(x).$$

Le but est de choisir $\mu(x)$ de telle sorte que :

$$\mu(x)\frac{dy}{dx} + \mu(x)P(x)y = \frac{d}{dx}[\mu(x)y].$$

D'où :

$$\frac{d}{dx}[\mu(x)y] = \mu(x)\frac{dy}{dx} + y\frac{d\mu}{dx}\mu(x)$$

Il faut donc que :

$$\frac{d\mu(x)}{dx} = \mu(x)P(x)$$

ce qui est une équation à variables séparables :

$$\frac{1}{\mu(x)}\, d(\mu(x)) = P(x)\, dx,$$

En intégrant les deux membres :

$$\int \frac{1}{\mu(x)}\, d(\mu(x)) = \int P(x)\, dx,$$

$$\ln[\mu(x)] = \int P(x)\, dx.$$

$$\ln[\mu(x)] = \int P(x)\, dx,$$

$$\mu(x) = e^{\int P(x)\, dx}.$$

Le facteur d'intégration peut être obtenu si la fonction $P(x)$ est intégrable. Après avoir trouvé $\mu(x)$, l'EDO se réduit à :

$$\frac{d}{dx}[\mu(x)y] = \mu(x)Q(x)$$

Et la solution est :

$$\mu(x)y = \int \mu(x)Q(x)\, dx$$

$$y(x) = \frac{1}{\mu(x)} \int \mu(x)Q(x)\, dx$$

1.4.3 Cas d'équation hétérogène

Il s'agit ici de l'équation $y'(t) = a(t)y(t) + b(t)$ avec une fonction b non nulle.
Le problème de Cauchy associé s'écrit :

$$\begin{cases} y'(t) = f(t, y(t)), & t \in [a, b] \\ y(a) = y_0 \end{cases}$$

Les fonctions a et b sont des fonctions continues de $[a, b]$ dans $\mathbb{R}$ et on cherche $y \in C^1(]t_0, T[, \mathbb{R}) \cap C([a, b], \mathbb{R})$ solution de (1).
Nous allons décrire plusieurs méthodes pour trouver la solution générale de l'équation

$$y'(t) = a(t)y(t) + b(t)$$

Cette solution dépend d'un paramètre dont la valeur est choisie pour satisfaire la condition initiale $y(t_0) = y_0$, ce qui permet d'obtenir l'unique solution de (1).

Recherche d'une solution particulière

Pour la résolution d'une EDO linéaire, la méthode à privilégier est la recherche d'une solution particulière, car elle nécessite le moins de calculs.
On suppose avoir trouvé une solution particulière y_p de l'équation :

$$y(t) = a(t)y(t) + b(t)$$

On a donc

$$y_p'(t) = a(t)y_p(t) + b(t)$$

pour tout t. Soit y une autre solution de l'équation :

$$y(t) = a(t)y(t) + b(t)$$

on pose $z = y - y_p$ et on a donc, grâce à la linéarité de l'équation :

$$\begin{aligned} z' &= y' - y_p' \\ &= ay + b - ay_p - b \\ &= az. \end{aligned}$$

On a donc

$$z' = az$$

ce qui donne

$$z(t) = Ce^{A(t)}$$

où A est une primitive de a et C est un réel arbitraire. On en déduit que

$$y = y_p + Ce^{A(t)}$$

Autrement dit, la solution générale de l'équation

$$y(t) = a(t)y(t) + b(t)$$

s'écrit comme la somme d'une solution particulière (notée ici y_p) et de la solution générale de l'équation homogène associée (équation $y' = a(t)y(t)$).
Si une solution particulière y_p est facile à trouver, le problème est alors résolu : la solution générale de l'équation (1) est

$$y(t) = Ce^{A(t)} + y_p(t)$$

où A est une primitive de a.

Exemple (Solution particulière évidente)

Si les fonctions a et b sont constantes,

$$a(t) = a \in \mathbb{R}^*$$

et

$$b(t) = b \in \mathbb{R}$$

pour tout t, alors $y_p(t) = -\frac{b}{a}$ est une solution particulière car :

$$y_p' = 0$$

et

$$ay_p + b = 0$$

.

La solution générale de l'équation $y' = ay + b$ est donc :

$$y(t) = Ce^{at} - \frac{b}{a}$$

1.5 Conclusion

Ce chapitre a permis d'établir les bases théoriques et pratiques des équations différentielles ordinaires. Nous avons vu qu'une EDO peut être caractérisée par son ordre, sa linéarité et sa forme normale, et que sous certaines conditions (théorème de Cauchy-Lipschitz), une solution unique existe.

Les méthodes analytiques, telles que la séparation des variables et le facteur intégrant, offrent des solutions exactes pour des cas particuliers, mais de nombreuses équations nécessitent des approches numériques (comme les méthodes d'Euler ou de Runge-Kutta) lorsque leur résolution explicite est impossible. Le choix du pas d'intégration et la stabilité des schémas numériques sont alors des aspects cruciaux pour obtenir des approximations fiables.

En résumé, la maîtrise des EDO repose à la fois sur une compréhension théorique des solutions et sur l'application de techniques adaptées, qu'elles soient exactes ou numériques. Ces concepts seront essentiels pour aborder des problèmes plus complexes dans les chapitres suivants, notamment ceux impliquant des systèmes d'équations ou des équations aux dérivées partielles.

Chapitre 2 : Méthodes numérique pour la résolution des EDO

Chapitre 2

Méthodes numériques pour la résolution des EDO

2.1 Introduction

Les méthodes numériques constituent un outil indispensable pour résoudre des équations différentielles qui ne possèdent pas de solutions analytiques explicites. Ce chapitre se concentre sur les techniques fondamentales de discrétisation et d'intégration numérique, en particulier la méthode d'Euler et les méthodes de Runge-Kutta.

Nous commencerons par définir le principe de discrétisation d'un intervalle et son importance dans la transformation d'un problème continu en un problème discret. Ensuite, nous détaillerons la méthode d'Euler, une approche simple mais fondamentale, avant d'aborder les méthodes plus avancées de Runge-Kutta (d'ordre 2 et 4), qui offrent une meilleure précision et stabilité.

L'objectif de ce chapitre est de fournir une compréhension claire des algorithmes pas à pas utilisés pour approximer les solutions d'équations différentielles, tout en mettant en lumière leurs avantages, leurs limites et leurs domaines d'application.

2.1.1 Définition et domaines d'application

Les équations différentielles ordinaires se rencontrent dans tous les domaines de la physique (électricité, mécanique, thermique...). Elles expriment une relation entre une fonction inconnue et ses dérivées, la fonction ne dépendant que d'une seule variable indépendante. Une équation différentielle ordinaire (EDO) est donc une relation entre :

— une variable indépendante ;
— une fonction ;
— et un certain nombre de dérivées de cette fonction.

2.2 Discrétisation de l'intervalle

2.2.1 Principe de la méthode

La discrétisation de l'intervalle est une étape fondamentale dans les méthodes numériques utilisées pour approximer les solutions d'équations différentielles, d'intégrales ou dans les problèmes de simulation numérique. Elle consiste à remplacer un intervalle continu par un ensemble fini de points. Cela permet de transformer un problème continu en un problème discret, que l'on peut résoudre à l'aide d'un ordinateur.
Pour la résolution numérique des EDO, la discrétisation de l'intervalle continu est une étape clé. Cela consiste à remplacer l'intervalle par un ensemble fini de points. Une EDO

d'ordre n peut être ramenée à sa forme normale :

$$y^{(n)} = f(t, y, y', \ldots, y^{(n-1)})$$

C'est cette forme qui est ensuite traitée numériquement en discrétisant l'intervalle de temps.

2.2.2 Formulation mathématique

Les équations du premier ordre

Sous sa forme la plus générale, une équation différentielle du premier ordre, dont la variable indépendante est t, s'écrit :

$$f(t, y(t), y'(t)) = 0$$

On peut être amené à chercher une solution sur un intervalle de temps donné $\mathcal{I}$, ou sur le plus grand intervalle possible où la solution reste définie. On considère d'abord cette équation comme une équation en $y'(t)$, et l'on cherche à la résoudre. Sous certaines conditions suffisantes (comme le théorème de *Cauchy-Lipschitz*), cette résolution est possible. L'équation peut alors être réécrite sous sa **forme normale** :

$$y'(t) = f(t, y(t))$$

Pour une équation du premier ordre :

$$\frac{dy}{dt} = f(t, y)$$

Forme normale :

$$y' = f(t, y)$$

Équations à variables séparables

Soient $f : I \to \mathbb{R}$ et $g : J \to \mathbb{R}$ deux fonctions continues. Une équation différentielle à variables séparables est de la forme :

$$\frac{dy}{dx} = \frac{f(x)}{g(y)}$$

Ce type d'équation est généralement simple à résoudre, car on peut séparer les variables et intégrer de part et d'autre :

$$g(y)\, dy = f(x)\, dx$$

$$\frac{dy}{dt} = g(t)h(y) \Rightarrow \frac{1}{h(y)} dy = g(t)dt$$

Équations linéaires d'ordre 1

Les équations différentielles linéaires du premier ordre sont de la forme :

$$y' = p(x)y + q(x), \quad \text{avec} \quad y' = \frac{dy}{dx}$$

Théorème 1.1 : La solution générale de l'équation s'écrit comme la somme de la solution générale de l'équation homogène et d'une solution particulière :

$$y = y_h + y_p$$

où :

— y_h est la solution générale de l'équation homogène associée $y' = p(x)y$,
— y_p est une solution particulière de l'équation complète.

$$\frac{dy}{dt} + p(t)y = q(t)$$

$$y(t) = \mu(t)^{-1}\left(\int \mu(t)q(t)\,dt + C\right), \quad \text{où } \mu(t) = e^{\int p(t)\,dt}$$

Équation différentielle linéaire du second ordre : Les équations différentielles linéaires du second ordre s'écrivent sous la forme suivante :

$$\frac{d^2y}{dx^2} + p(x)\frac{dy}{dx} + q(x)y = r(x)$$

où $p(x)$, $q(x)$ et $r(x)$ sont des fonctions continues données. **Définition :**
— Si $r(x) = 0$, l'équation est dite **homogène**.
— Si $r(x) \neq 0$, elle est dite **non homogène**.
Théorème :
La solution générale de l'équation non homogène s'écrit sous la forme :

$$y(x) = y_h(x) + y_p(x)$$

où :
— $y_h(x)$ est la solution générale de l'équation homogène associée :

$$\frac{d^2y}{dx^2} + p(x)\frac{dy}{dx} + q(x)y = 0$$

— $y_p(x)$ est une solution particulière de l'équation complète :

$$\frac{d^2y}{dx^2} + p(x)\frac{dy}{dx} + q(x)y = r(x)$$

$$y'' + p(x)y' + q(x)y = r(x)$$

Si $r(x) = 0$: homogène. Théorème :

$$y = y_h + y_p$$

2.2.3 Algorithme pas à pas

Le concept fondamental de cet algorithme repose sur l'idée que si l'on connaît la valeur d'une fonction y à un point x, ainsi que son taux de variation (sa dérivée $f(x, y)$) à ce point, on peut estimer la valeur de la fonction à un point légèrement plus éloigné. C'est l'essence même de l'approximation par développement limité d'ordre 1 (similaire à la méthode d'*Euler*).

L'algorithme procède comme suit :

1. **Initialisation (Lignes 1-3) :** On commence avec une condition initiale connue, x_0 et y_0. Ces valeurs représentent le point de départ de la solution de l'EDO.
2. **Itération (Boucle "for", Lignes 4-8) :** L'algorithme répète une série de calculs pour un nombre de pas spécifié (n). À chaque pas (i de 1 à n) :
   — **Calcul de la nouvelle valeur de y (Ligne 5) :** La nouvelle valeur de y est estimée en ajoutant au y actuel le produit du pas de temps (h) par la valeur de la dérivée ($f(x, y)$) au point (x, y) actuel. Cette étape peut être vue comme une extrapolation linéaire de la solution.

— **Mise à jour de x (Ligne 6) :** Le point x est avancé d'un pas de h, ce qui correspond à la progression dans la variable indépendante.
— **Affichage/Enregistrement (Ligne 7) :** Les paires (x, y) sont affichées ou enregistrées, fournissant ainsi les points de la solution numérique approximée.

Algorithm 1 Méthode pas à pas générale

1: **Entrée** : $f(x, y), x_0, y_0, h, n$
2: Initialiser $x \leftarrow x_0$
3: Initialiser $y \leftarrow y_0$
4: **for** $i = 1$ **à** n **do**
5: Calculer la nouvelle valeur de y : $y \leftarrow y + h \cdot f(x, y)$
6: Mettre à jour x : $x \leftarrow x + h$
7: Afficher ou enregistrer les résultats (x, y)
8: **end for**

2.3 Méthode d'Euler

2.3.1 Principe de la méthode

Les méthodes d'*Euler* sont des méthodes numériques classiques à un pas, permettant, à partir d'une condition initiale, de calculer les valeurs approchées de la solution à chaque pas h.

Pour approcher la solution $y(t)$ sur l'intervalle $t \in [a, b]$, on choisit $n+1$ points distincts $t_0, t_1, t_2, \ldots, t_n$, avec $t_0 = a$ et $t_n = b$. Le pas de discrétisation est défini par :

$$h = \frac{b - a}{n}$$

La résolution numérique consiste à discrétiser l'axe des abscisses comme suit :

$$t_n = t_0 + nh \quad (n \in \mathbb{N})$$

On cherche alors u_n comme approximation de y au point t_n, c'est-à-dire :

$$y(t) \approx y(t_n) \equiv u_n$$

Ainsi, l'ensemble des approximations successives $\{u_0, u_1, u_2, \ldots, u_n\}$,

Ces méthodes étant itératives, la suite $\{u_n\}_{n \in \mathbb{N}}$ doit être initialisée à partir d'une condition initiale afin de pouvoir calculer ses termes suivants.

$$h = \frac{b - a}{n}, \quad t_n = t_0 + nh$$

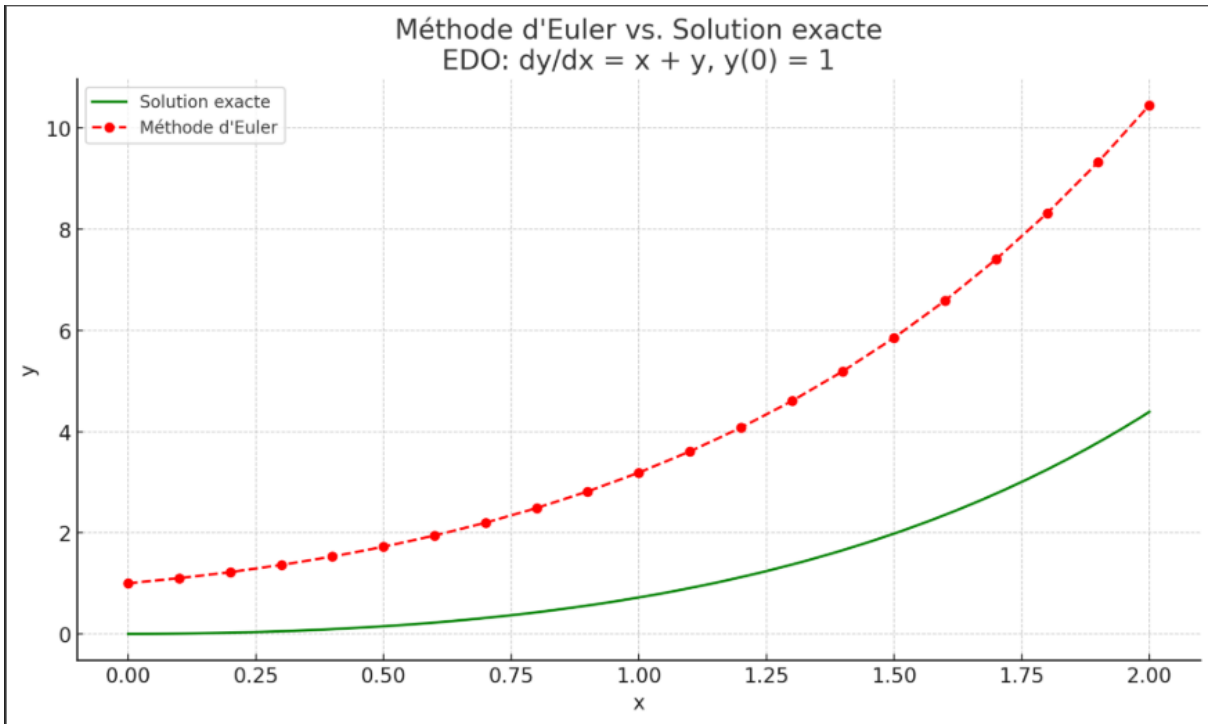

FIGURE 2.1 – Méthode d'*Euler* vs. Solution exacte pour EDO : $dy/dx = x + y, y(0) = 1$

2.3.2 Formulation mathématique

Considérons une équation différentielle du premier ordre donnée par :

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0$$

où $f(x, y)$ est une fonction continue définie sur un certain domaine, x_0 est la condition initiale, et y_0 est la valeur de la solution en x_0. Cette équation constitue le point de départ pour l'application de méthodes numériques telles que la méthode d'*Euler*. **Équation différentielle à résoudre :**

$$y' = f(t, y)$$

Méthodes d'*Euler* : principes et formulations

Méthode d'*Euler* explicite

Cette méthode consiste à approximer la dérivée à l'aide d'un développement limité. Considérons l'équation différentielle :

$$y'(t) = f(t, y(t)) \quad \Leftrightarrow \quad \int_{t_n}^{t_{n+1}} y'(t)\, dt = \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt$$

Trouver la solution de cette équation revient à calculer l'intégrale de $f(t, y(t))$ entre t_n et t_{n+1}, soit :

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt$$

Cette relation peut être approximée numériquement à l'aide de $u_n \approx y(t_n)$ et $u_{n+1} \approx y(t_{n+1})$. En utilisant par exemple la méthode des rectangles à gauche pour approximer l'intégrale, on obtient le schéma numérique suivant :

$$u_{n+1} = u_n + h\, f(t_n, u_n)$$

C'est la formule clé de la méthode d'*Euler* explicite.

Il s'agit du schéma d'*Euler* explicite, aussi appelé schéma d'*Euler* progressif.

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

Cette méthode approxime l'intégrale suivante :

$$\int_{t_n}^{t_{n+1}} f(t, y(t))\, dt \approx h \cdot f(t_n, y_n)$$

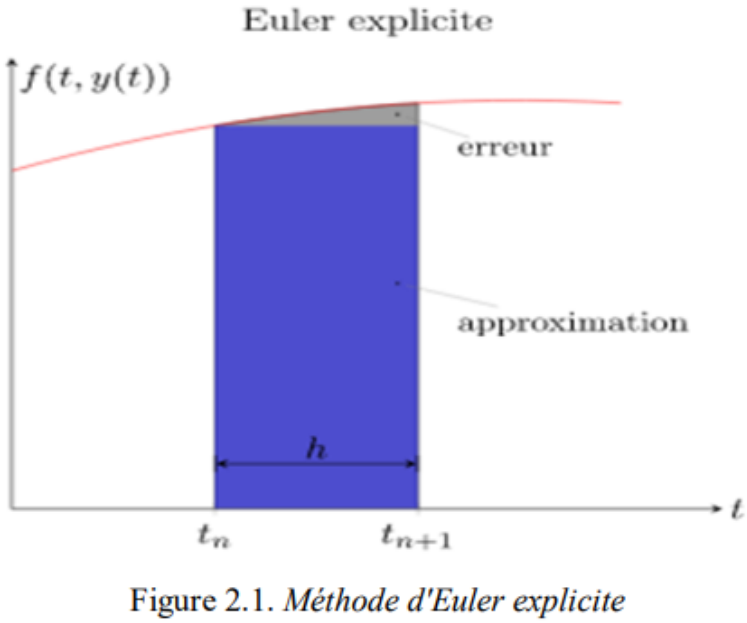

FIGURE 2.2 – Représentation graphique de la méthode d'*Euler* explicite : approximation par rectangle à gauche.

Méthode d'*Euler* implicite

Cette méthode consiste à approximer la dérivée à l'aide d'un développement limité rétrograde. Le schéma numérique obtenu est donné par :

$$u_{n+1} = u_n + h\, f(t_{n+1}, u_{n+1}) \quad \forall n \in \mathbb{N}, \quad y(a) = y_0 = u_0$$

Ce schéma nécessite la résolution d'une équation à chaque itération, ce qui entraîne des calculs plus longs, mais il offre généralement une meilleure stabilité numérique, notamment pour les équations raides.

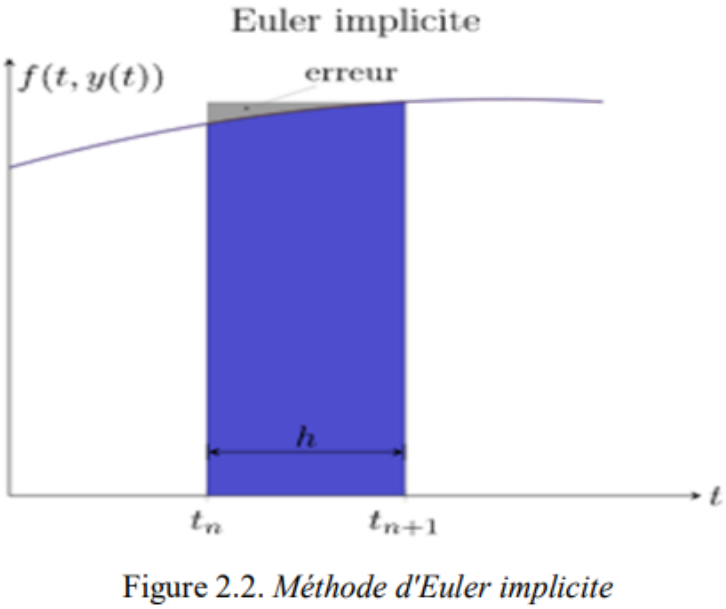

FIGURE 2.3 – Représentation graphique de la méthode d'*Euler* implicite : approximation par rectangle à droite.

Méthode d'*Euler* modifiée

En utilisant la méthode du point milieu pour approximer l'intégrale, on obtient le schéma numérique suivant :

$$u_{n+1} = u_n + h\, f\left(t_n + \frac{h}{2}, u_{n+\frac{1}{2}}\right), \quad \forall n \in \mathbb{N}$$

avec :

$$u_{n+\frac{1}{2}} \approx u_n + \frac{h}{2} f(t_n, u_n), \quad y(t_0) = y_0 = u_0$$

Analyse de l'erreur : Concernant la précision des méthodes numériques, la méthode d'*Euler* modifiée se distingue par son ordre 2, offrant ainsi une meilleure convergence. En revanche, la méthode d'*Euler* classique, d'ordre 1, implique que l'erreur sur la solution $y(t)$ à un point t donné diminue proportionnellement à $\frac{1}{n}$, où n représente le nombre de pas d'intégration. Tel qu'illustré par la différence significative est observée entre la solution exacte et l'approximation obtenue par la méthode d'*Euler*, cette divergence étant d'autant plus prononcée sur des intervalles de temps étendus ou avec des pas de discrétisation de grande taille. Cette constatation met en lumière l'aspect crucial du choix du pas h et de l'ordre de la méthode pour assurer la fiabilité et la précision de la solution numérique.

2.3.3 Algorithme pas à pas de la méthode d'*Euler*

Concept de l'algorithme pas à pas générale

Explication de l'algorithme de la méthode d'Euler L'algorithme de la méthode d'Euler explicite permet d'approcher numériquement la solution d'une équation différentielle ordinaire (EDO) de la forme

$$\frac{dy}{dt} = f(t, y),$$

avec une condition initiale donnée

$$y(a) = y_0.$$

Ce type d'équation modélise souvent l'évolution d'un phénomène dans le temps ou l'espace.

Le principe de la méthode consiste à avancer pas à pas à partir du point initial, en utilisant la pente locale (c'est-à-dire la dérivée $f(x, y)$) pour estimer la valeur suivante de y. Cette estimation est effectuée à l'aide d'une formule simple :

$$y_{i+1} = y_i + h \cdot f(x_i, y_i),$$

où h est un petit pas de progression choisi à l'avance, et

$$x_{i+1} = x_i + h.$$

L'algorithme commence par initialiser les variables x et y avec les valeurs de départ x_0 et y_0. Ensuite, il effectue une boucle qui se répète n fois, où à chaque itération, les nouvelles valeurs de x et y sont calculées à l'aide de la formule ci-dessus. Après chaque mise à jour, les valeurs de x et y sont affichées ou stockées.

Cette méthode est simple à mettre en œuvre et permet d'obtenir rapidement une solution approximative, surtout lorsque la solution exacte de l'équation différentielle est inconnue ou difficile à obtenir. Cependant, la précision de la méthode dépend fortement de la taille du pas h : plus il est petit, plus l'approximation est précise, mais cela nécessite plus d'itérations.

Algorithm 2 Algorithme de la méthode d'*Euler* explicite

$f(x, y),\ x_0,\ y_0,\ h,\ n$ Les valeurs approchées de y Initialiser $x \leftarrow x_0$
Initialiser $y \leftarrow y_0$
Afficher `"x"`, `"y"`

for $i \leftarrow 1\ n$ **do** $y \leftarrow y + h \cdot f(x, y)$
$x \leftarrow x + h$
Afficher `"x"`, `"y"`

2.4 Méthode de Runge-Kutta

Les méthodes de Runge-Kutta (*Runge-Kutta d'ordre 2*, *Runge-Kutta d'ordre 4*) sont des schémas numériques à un pas pour résoudre des EDO du type :

$$y' = f(t, y)$$

avec condition initiale :

$$y(a) = y_0$$

Elles ont été introduites par Carl Runge et Martin Kutta au début du XXe siècle et sont aujourd'hui très populaires grâce à leur simplicité de mise en œuvre et leur bonne précision[1] .

2.4.1 Principe de la méthode

Les méthodes de Runge–Kutta généralisent la méthode du point milieu en évaluant plusieurs pentes intermédiaires pour affiner l'approximation :

$$Y_0(b) \approx \frac{Y_{n+1} - Y_n}{h} = \Phi(t_n, Y_n, h)$$

Cette approximation est affinée à l'aide de plusieurs points intermédiaires $(t_{n,i}, Y_{n,i})$. On pose :

$$t_{n,i} = t_n + c_i h, \quad 1 \leq i \leq q, \quad c_i \in [0,1].$$

À chacun de ces points, on définit la pente :

$$p_{n,i} = f(t_{n,i}, Y_{n,i})$$

où les valeurs sont obtenues successivement par :

$$Y_{n,i} = Y_n + h \sum_{j=1}^{i-1} a_{i,j} p_{n,j}$$

Finalement, on obtient :

$$Y_{n+1} = Y_n + h \sum_{j=1}^{q} b_j p_{n,j}$$

2.4.2 Runge-Kutta d'ordre 2 - Algorithme

La méthode de *Runge-Kutta d'ordre 2 (RK2)* constitue une famille de schémas numériques explicites permettant de résoudre une équation différentielle ordinaire (EDO) avec une précision supérieure à celle de la méthode d'Euler. Elle repose sur une combinaison pondérée de plusieurs évaluations de la fonction dérivée f au sein de l'intervalle $[t_n, t_{n+1}]$.

Sous sa forme générale, pour un paramètre $\alpha \in (0,1]$, la méthode RK2 peut s'écrire de façon condensée :

$$\Phi(t_n, Y_n, h) = \left(1 - \frac{1}{2\alpha}\right) F(t_n, Y_n) + \frac{1}{2\alpha} F\big(t_n + \alpha h, Y_n + \alpha h F(t_n, Y_n)\big),$$

où l'approximation suivante est alors donnée par :

$$Y_{n+1} = Y_n + h\,\Phi(t_n, Y_n, h).$$

Elles ont été introduites par Carl Runge et Martin Kutta au début du XXe siècle et sont aujourd'hui très populaires grâce à leur simplicité de mise en œuvre et leur bonne précision [2].

Propriété :

On peut citer les cas particuliers importants suivants :

- Pour $\alpha = \frac{1}{2}$, on retrouve la méthode du point milieu

$$Y_{n+1} = Y_n + h\,F\left(t_n + \frac{h}{2}, Y_n + \frac{h}{2} F(t_n, Y_n)\right),$$

qui est basée sur la méthode d'intégration du point milieu :

$$\int_0^1 g(t)\,dt \approx g\left(\frac{1}{2}\right).$$

- Pour $\alpha = 1$, on obtient la méthode de Heun

$$Y_{n+1} = Y_n + h\left(\frac{1}{2} F(t_n, Y_n) + \frac{1}{2} F\left(t_{n+1}, Y_n + hF(t_n, Y_n)\right)\right),$$

basée sur la méthode d'intégration des trapèzes :

$$\int_0^1 g(t)\,dt \approx \frac{1}{2}\left(g(0) + g(1)\right).$$

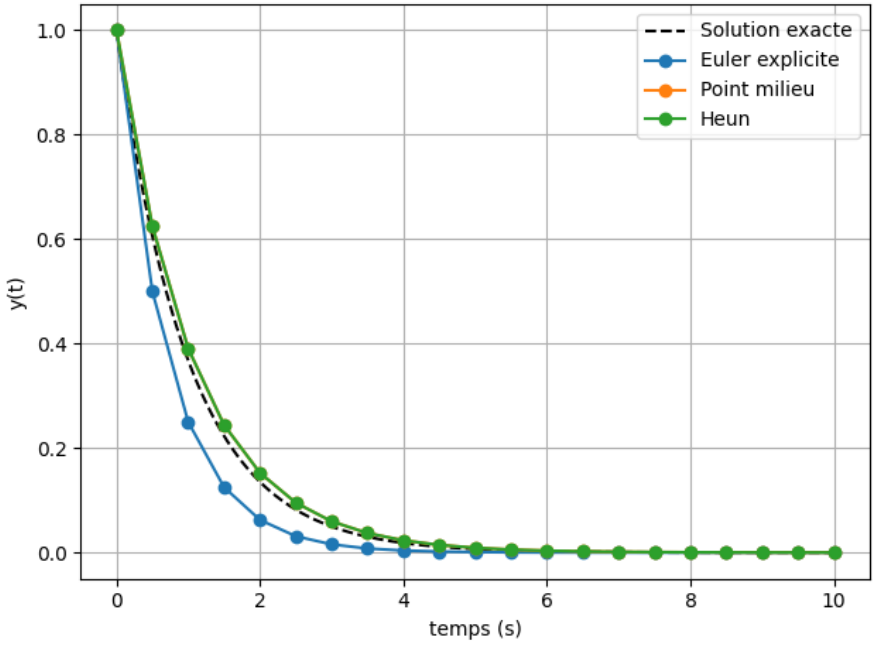

FIGURE 2.4 – Comparaison de différentes méthodes Runge-Kutta d'ordre 2 pour $y' = -y$, $y(0) = 1$.

Les courbes représentent différentes méthodes RK2 pour la résolution de l'équation différentielle $y' = -y$ avec la condition initiale $y(0) = 1$, pour un pas fixe $h = 0{,}5$.

$$y_{n+1} = y_n + h\left(a_1 k_1 + a_2 k_2\right), \qquad \begin{cases} k_1 = f(t_n, y_n) \\ k_2 = f(t_n + \alpha h,\ y_n + \beta h k_1) \end{cases}$$

Le tableau suivant présente les coefficients utilisés dans chaque méthode : a_1, a_2, α et β, qui définissent la forme générale du schéma à deux étapes :

Méthode	a_1	a_2	α	β
Euler explicite	1	0	0	0
Point milieu	0	1	$\frac{1}{2}$	$\frac{1}{2}$
Heun	$\frac{1}{2}$	$\frac{1}{2}$	1	1

TABLE 2.1 – Paramètres des méthodes numériques

Algorithm 3 Méthode générale Runge-Kutta d'ordre 2 (RK2)

```
Initialiser le pas h et la durée T
Initialiser les conditions initiales : t = 0 et y = y(0)
Définir la fonction f(t, y)
Fixer les paramètres α, β, a1, a2
while t ≤ T do
    Calculer k1 = f(t, y)
    Calculer k2 = f(t + αh, y + βh · k1)
    Mettre à jour : y ← y + h(a1 k1 + a2 k2)
    Mettre à jour : t ← t + h
    Enregistrer les données
end while=0
```

2.4.3 Runge-Kutta d'ordre 4 - Algorithme

La méthode de *Runge-Kutta d'ordre 4 (RK4)* est un schéma numérique explicite classique, largement utilisé pour l'approximation des solutions d'équations différentielles ordinaires.

La mise à jour de la méthode RK4 se fait à l'aide des quatre pentes intermédiaires définies par :

$$y_{n+1} = y_n + h\left[\frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6}\right]$$

avec :

$$\begin{cases} k_1 = f(t_n, y_n), \\ k_2 = f\left(t_n + \dfrac{h}{2}, y_n + \dfrac{h}{2}k_1\right), \\ k_3 = f\left(t_n + \dfrac{h}{2}, y_n + \dfrac{h}{2}k_2\right), \\ k_4 = f(t_n + h, y_n + hk_3). \end{cases}$$

Par rapport à la méthode RK2, ce schéma numérique nécessite deux fois plus de calculs à chaque itération, ce qui entraîne un temps de calcul plus long, sans oublier les erreurs d'arrondi qui s'accumulent plus rapidement. Cependant, cet inconvénient est compensé par un gain de précision. En effet, comme son nom l'indique, il s'agit d'une méthode d'ordre 4[2].

On peut le vérifier sur un exemple simple. Prenons l'équation :

$$\dot{y}(t) = -y(t) \quad \text{avec} \quad y(0) = 1$$

dont la solution est la fonction exponentielle :

$$y(t) = e^{-t}$$

Quelle valeur prévoit la méthode de Runge-Kutta-4 pour $y_1 = y(h)$? Sachant que l'on a ici $f(t, y(t)) = -y(t)$, les coefficients de la méthode s'écrivent alors :

$$\begin{aligned} k_1 &= f(0, y_0) = -y_0 \\ k_2 &= f\left(\frac{h}{2}, y_0 + \frac{h}{2}k_1\right) = -\left(y_0 + \frac{h}{2}k_1\right) \\ k_3 &= f\left(\frac{h}{2}, y_0 + \frac{h}{2}k_2\right) = -\left(y_0 + \frac{h}{2}k_2\right) \\ k_4 &= f(h, y_0 + hk_3) = -\left(y_0 + hk_3\right) \end{aligned}$$

d'où il vient :

$$y_1 = y_0 + h\left[\frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4\right]$$

qui, après substitution des valeurs de k_1, k_2, k_3, et k_4, donne :

$$y_1 = y_0 + h - \frac{h^2}{2} + \frac{h^3}{6} - \frac{h^4}{24}$$

qui n'est rien d'autre que le développement limité de la solution exacte $y(h)$ à l'ordre 4. Ainsi, même avec un pas modeste, la précision reste bonne[1].

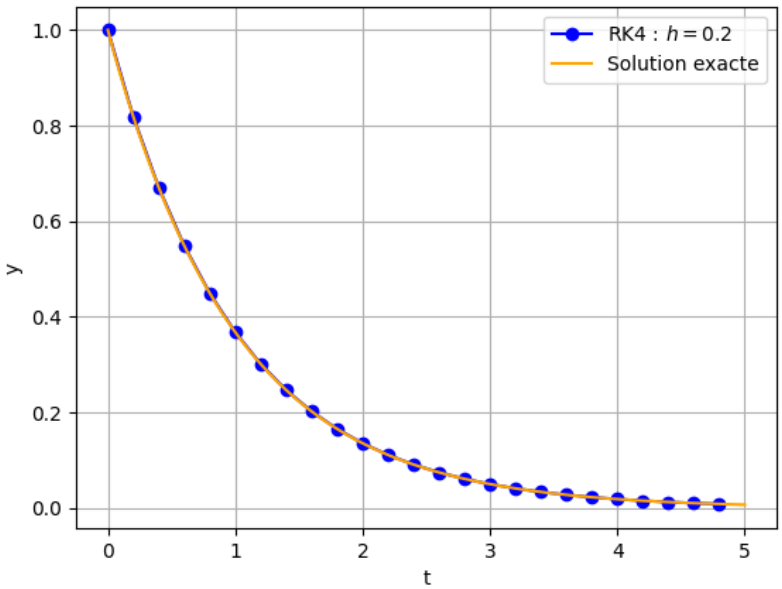

FIGURE 2.5 – Exemple d'intégration par la méthode de Runge-Kutta-4.

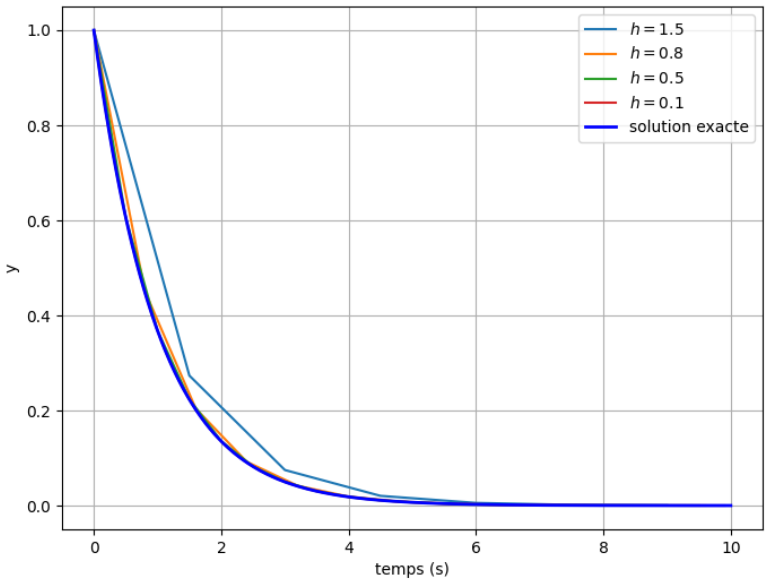

FIGURE 2.6 – Résolution numérique de $y^{\cdot}(t) + y(t) = 0$ avec $y(0) = 1$ par le schéma classique Runge-Kutta d'ordre 4

2.4.4 Erreur de Runge–Kutta 4

Soit l'équation différentielle ordinaire

$$y' = f(t, y), \quad y(a) = y_0,$$

avec $f \in C^5([a, b] \times \mathbb{R})$.

La méthode de Runge–Kutta d'ordre 4 (RK4) utilise un pas h pour approximer la solution.

L'erreur locale de troncature (LTE) au pas t_n est donnée par :

$$\mathrm{LTE}_n = y(t_{n+1}) - y_{n+1} = Ch^5 + O(h^6),$$

où C est une constante dépendant des dérivées de f jusqu'au quatrième ordre.

En cumulant ces erreurs sur $N \approx \frac{T}{h}$ pas, l'erreur globale à l'instant final T satisfait :

$$|y(T) - y_N| = O(h^4).$$

Autrement dit, diminuer le pas h par deux réduit l'erreur globale d'environ un facteur $2^4 = 16$.

Cette propriété fait de RK4 une méthode à convergence rapide, efficace pour des problèmes nécessitant une précision importante sans coûts excessifs[4].

Algorithm 4 Méthode de Runge-Kutta d'ordre 4 (RK4)

```
Initialisation : Pas h, durée T
Initialiser t = 0 et y = y(0)
Définir la fonction f(t, y)
while t ≤ T do
    Calculer k1 = f(t, y)
    Calculer k2 = f(t + h/2, y + h/2 k1)
    Calculer k3 = f(t + h/2, y + h/2 k2)
    Calculer k4 = f(t + h, y + h k3)
    Mettre à jour y = y + h/6 (k1 + 2k2 + 2k3 + k4)
    Mettre à jour t = t + h
    Enregistrer les données
end while
```

2.4.5 Exemple Méthode de Heun

Si on regarde notre système

$$\begin{cases} y_1 + y_2 = 1 \\ \alpha_1 \cdot y_2 = \dfrac{1}{2} \\ \beta_{1,1} \cdot y_2 = \dfrac{1}{2} \end{cases}$$

cette méthode correspond au cas où $\gamma_2 = \dfrac{1}{2}$ et donc, $\gamma_1 = \dfrac{1}{2}$, $\alpha_1 = 1$ et $\beta_{1,1} = 1$. Cette méthode se formule comme suit :

$$C = y_n + \frac{h}{2}\left[f(x_n, y_n) + f\left(x_n + h, y_n + hf(x_n, y_n)\right)\right]$$

Point de vue géométrique

On veut connaître y_{n+1} en $x_{n+1} = x_n + h$ connaissant (x_n, y_n) et $f(x, y) = \nu$.

1. On considère la pente en x_n, c'est-à-dire la tangente. On la prolonge pour avoir la valeur en x_{n+1}. Ceci correspond à la *méthode d'Euler*.
2. On considère ensuite la pente en x_{n+1}. On prend la parallèle de cette pente passant par (x_n, y_n). On la prolonge pour obtenir la valeur en x_{n+1}.
3. La méthode de Heun correspond à la moyenne de ces deux pentes. On prend le nouveau point y_{n+1} comme l'image de x_{n+1} sur la droite obtenue.

Méthode de la tangente améliorée

À nouveau, en regard avec notre système

$$\begin{cases} y_1 + y_2 = 1 \\ \alpha_1 \cdot y_2 = \dfrac{1}{2} \\ \beta_{1,1} \cdot y_2 = \dfrac{1}{2} \end{cases}$$

, cette méthode correspond au cas où $\gamma_2 = 1$ et donc, $\gamma_1 = 0$, $\alpha_1 = \dfrac{1}{2}$ et $\beta_{1,1} = \dfrac{1}{2}$. Cette méthode se formule de la façon suivante :

$$y_{n+1} = y_n + hf\left(\frac{h}{2} + x_n, y_n + \frac{h}{2} f(x_n, y_n)\right)$$

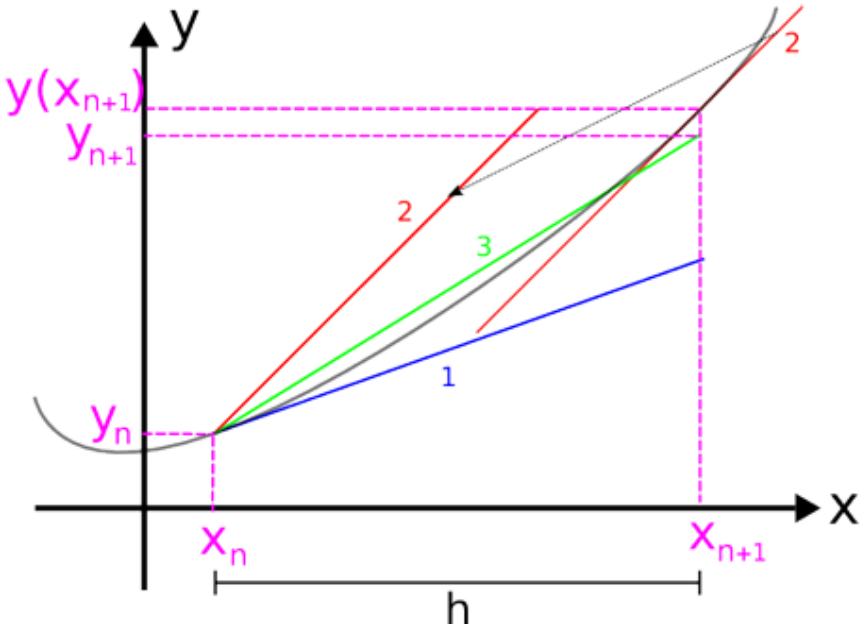

FIGURE 2.7 – methode de heun

Considérons le problème de Cauchy :

$$\begin{cases} y'(t) = f(t, y(t)), \quad t \in [a, b] \\ y(a) = y_0 \end{cases}$$

Lemma 1 (Lemme de Grönwall discret) Soit (u_n) une suite réelle vérifiant :

$$u_{n+1} \leq (1 + hL)u_n + h^2 M$$

Alors, pour $t_n = a + nh$ $(n \in [a, b])$, on a :

$$u_n \leq \left(u_0 + \frac{hM}{L}\right) e^{L(t_n - a)} - \frac{hM}{L} \quad \text{et si } u_0 = 0, \text{ alors} \quad u_n \leq \frac{hM}{L}(e^{L(t_n - a)} - 1)$$

Lemma 2 (Convergence de la méthode d'Euler explicite) Soit $f : [a, b] \times \mathbb{R} \to \mathbb{R}$ une fonction continue, lipschitzienne par rapport à la seconde variable avec une constante $L > 0$, et soit $y(t)$ une solution de classe $\mathcal{C}^2$ sur $[a, b]$.

On considère la méthode d'Euler explicite :

$$y_{n+1} = y_n + hf(t_n, y_n), \quad t_n = a + nh, \quad n = 0, 1, \ldots, N, \quad h = \frac{b - a}{N}$$

Alors, l'erreur globale entre la solution exacte $y(t_n)$ et l'approximation y_n obtenue par la méthode d'Euler satisfait :

$$\|y(t_n) - y_n\| \leq Ch, \quad \forall n \in \mathbb{N}$$

où $C = \frac{M}{2L}(e^{L(b-a)} - 1)$ et $M = \max_{t\in[a,b]} \|y''(t)\|$.

Par conséquent, la méthode d'Euler explicite est d'ordre de convergence global égal à 1.

Démonstration

Soit $e_n = y(t_n) - y_n$ l'erreur globale à l'étape n.

1. Erreur locale (consistance)

On utilise un développement de Taylor :

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(\xi_n) \quad \text{avec } \xi_n \in [t_n, t_{n+1}]$$

$$= y(t_n) + hf(t_n, y(t_n)) + \frac{h^2}{2}y''(\xi_n)$$

Comparée à l'itération d'Euler :

$$y_{n+1} = y_n + hf(t_n, y_n)$$

l'erreur locale de consistance est :

$$\tau_{n+1} = y(t_{n+1}) - [y(t_n) + hf(t_n, y(t_n))] = \frac{h^2}{2}y''(\xi_n) = \|\tau_{n+1}\| = \mathcal{O}(h^2)$$

2. Erreur globale (convergence)

Soit $e_n = y(t_n) - y_n$. On a :

$$\begin{aligned} e_{n+1} &= y(t_{n+1}) - y_{n+1} \\ &= y(t_{n+1}) - y(t_n) - hf(t_n, y(t_n)) + h[f(t_n, y(t_n)) - f(t_n, y_n)] + e_n \\ &= \underbrace{y(t_{n+1}) - y(t_n) - hf(t_n, y(t_n))}_{\tau_{n+1}} + \underbrace{h[f(t_n, y(t_n)) - f(t_n, y_n)]}_{\text{erreur de stabilité}} + e_n \end{aligned}$$

En norme :

$$\|e_{n+1}\| \leq \|e_n\| + hL\|e_n\| + \frac{h^2}{2}M = (1 + hL)\|e_n\| + \frac{h^2}{2}M$$

En utilisant le lemme de Grönwall discret par récurrence, $u_n := \|e_n\|$, avec $e_0 = 0$ (car $y_0 = y(a)$), on obtient :

$$\|e_n\| \leq \frac{h}{2}M \cdot \frac{e^{LT} - 1}{L} \leq Ch \Rightarrow \|e_n\| = \mathcal{O}(h)$$

avec constante explicite :

$$C = \frac{M}{L}(e^{L(b-a)} - 1)$$

— L'erreur locale est d'ordre $\mathcal{O}(h^2)$

— L'erreur globale est d'ordre $\mathcal{O}(h)$

On conclut donc que :

$$\|y(t_n) - y_n\| = \mathcal{O}(h)$$

La méthode d'Euler explicite est d'ordre de convergence global égal à 1.

Les équations différentielles ordinaires (EDO) sont fondamentales en modélisation mathématique. Lorsque leur résolution analytique est impossible, les méthodes numériques deviennent essentielles. Cette étude compare cinq approches classiques sur l'EDO linéaire :

$$\frac{dy}{dt} = -2y, \quad y(0) = 1, \quad t \in [0, 1]$$

dont la solution exacte est $y(t) = e^{-2t}$.

Mesure des erreurs : L'erreur absolue à $t = 1$ est calculée par :

$$\text{Erreur} = |y_{\text{num}}(1) - e^{-2}|$$

L'ordre de convergence expérimental P est estimé par :

$$p = \log_2 \left(\frac{\text{Erreur}(h)}{\text{Erreur}(h/2)} \right)$$

N	$h = 1/N$	Euler explicite		Euler implicite		RK2		RK4	
		Erreur	p	Erreur	p	Erreur	p	Erreur	p
10	0.1	0.126	–	0.118	–	0.012	–	6.2×10^{-5}	–
20	0.05	0.061	1.04	0.057	1.05	0.0031	1.95	3.9×10^{-6}	4.00
40	0.025	0.030	1.02	0.028	1.03	0.00078	1.99	2.4×10^{-7}	4.02
80	0.0125	0.015	1.01	0.014	1.01	0.00020	1.99	1.5×10^{-8}	4.00

Table 2.2 – Comparaison des erreurs et ordres de convergence pour différentes méthodes numériques

2.5 Conclusion

Ce chapitre a mis en évidence l'importance des méthodes numériques dans la résolution des équations différentielles ordinaires (EDO), notamment lorsque les solutions analytiques sont inaccessibles. Nous avons étudié les méthodes fondamentales telles que la méthode d'Euler, la méthode de Runge-Kutta (ordres 2 et 4), ainsi que la méthode de Heun. Chacune de ces méthodes repose sur le principe de discrétisation d'un intervalle, permettant de transformer un problème continu en un problème discret, plus facilement traitable par un ordinateur.

Les algorithmes pas à pas présentés montrent comment approximer une solution à partir d'une condition initiale, en tenant compte du choix du pas et de la stabilité numérique, éléments essentiels pour garantir la précision des résultats. Nous avons également abordé l'analyse des erreurs locales et globales, démontrant que des méthodes plus avancées comme Runge-Kutta d'ordre 4 offrent une précision nettement supérieure à celle d'Euler.

En somme, ce chapitre fournit les outils numériques indispensables pour aborder efficacement les problèmes où les solutions exactes ne peuvent être obtenues, et prépare à leur application dans des domaines variés, tels que la biologie, la physique ou l'ingénierie, traités dans le chapitre suivant.

Chapitre 3 : Applications dans les différents domaines

Chapitre 3

Application dans les different domaine

3.1 Introduction

Introduction

Les équations différentielles ordinaires (EDO) trouvent des applications concrètes dans de nombreux domaines scientifiques. Dans ce chapitre, nous nous concentrerons sur deux cas emblématiques : la biologie et la physique.

En biologie, les EDO permettent de modéliser l'évolution d'épidémies à travers des systèmes dynamiques tels que le modèle SIR, qui divise la population en individus sensibles, infectés et guéris. Ce modèle offre un cadre mathématique pertinent pour étudier la propagation des maladies infectieuses et évaluer l'impact des interventions sanitaires.

En physique, les EDO interviennent dans l'analyse du mouvement, notamment dans la modélisation de la chute libre d'un corps soumis à des forces comme la gravité et la résistance de l'air. Ce phénomène, bien qu'élémentaire, illustre parfaitement comment les équations différentielles traduisent les lois fondamentales de la mécanique classique.

Ainsi, à travers ces deux exemples, ce chapitre mettra en évidence l'utilité des méthodes numériques pour simuler, analyser et interpréter des phénomènes complexes issus du monde réel.

Objectif du chapitre Ce chapitre vise à illustrer l'intérêt et la puissance des approches numériques pour la résolution des EDO à travers des applications concrètes. Nous nous focaliserons notamment sur la modélisation épidémiologique avec le modèle SIR, permettant ainsi de mettre en évidence l'importance de ces méthodes dans des contextes réels et d'actualité.

3.2 Biologie – Modèle SIR

3.2.1 Introduction au modèle SIR

Contexte épidémiologique Les épidémies de maladies infectieuses (grippe, COVID-19, variole, etc.) représentent un enjeu majeur de santé publique. La compréhension de leur propagation est essentielle pour prendre des décisions éclairées. À cet effet, les modèles mathématiques comme le modèle SIR permettent de prédire et de simuler les dynamiques d'une épidémie, facilitant ainsi la planification des interventions (confinement, vaccination...).

Présentation du modèle SIR Le modèle SIR est un modèle compartimental dans lequel la population est divisée en trois groupes :

— S (Susceptibles) : individus sains susceptibles d'être infectés.
— I (Infectieux) : individus infectés, capables de transmettre la maladie.
— R (Rétablis ou Retirés) : individus qui ne peuvent plus transmettre la maladie, soit parce qu'ils sont guéris et immunisés, soit parce qu'ils sont décédés ou isolés.

Hypothèses simplificatrices
— La population totale N est constante (pas de naissance ni de migration).
— La transmission se fait par contacts aléatoires homogènes.
— L'immunité après la guérison est acquise à vie.
— La durée de la maladie est constante (pas de variations individuelles).
— Pas de prise en compte explicite des décès naturels.

Objectif de l'application Ce modèle, bien que simple, permet :
— De décrire l'évolution de l'épidémie.
— D'étudier les effets de mesures de santé publique.
— D'utiliser des méthodes numériques pour simuler différents scénarios.

3.2.2 Les équations différentielles associées

Le système d'équations différentielles ordinaires (EDO) du modèle SIR s'écrit :

$$\begin{cases} \frac{dS}{dt} = -\beta SI \\ \frac{dI}{dt} = \beta SI - \gamma I \\ \frac{dR}{dt} = \gamma I \end{cases}$$

Interprétation des termes
— $\frac{dS}{dt} = -\beta SI$: diminution des susceptibles par infection.
— $\frac{dI}{dt} = \beta SI - \gamma I$: évolution des infectés (nouveaux cas moins les guérisons).
— $\frac{dR}{dt} = \gamma I$: augmentation des rétablis.

Paramètres du modèle
— β : taux de transmission.
— γ : taux de guérison ($1/\gamma$ est la durée moyenne de la maladie).

Conditions initiales

$$S(0) = S_0, \quad I(0) = I_0, \quad R(0) = R_0, \quad \text{avec } S(t) + I(t) + R(t) = N$$

3.2.3 La résolution numérique

Pourquoi la résolution numérique ? Dans la majorité des cas, le système d'équations différentielles du modèle SIR :

$$\begin{cases} \dfrac{dS}{dt} = -\beta SI \\ \dfrac{dI}{dt} = \beta SI - \gamma I \\ \dfrac{dR}{dt} = \gamma I \end{cases}$$

n'admet pas de solution analytique explicite. En effet, le couplage non linéaire entre les variables rend la résolution exacte très complexe, voire impossible, pour des cas réalistes. C'est pourquoi il est nécessaire d'employer des méthodes numériques pour approximer la

solution de manière fiable et efficace. Ces méthodes permettent de simuler le comportement du système, d'en analyser la dynamique, et d'explorer différents scénarios (variation des paramètres, politiques de santé, etc.).

Méthodes numériques utilisées Plusieurs méthodes numériques peuvent être utilisées pour résoudre un système d'EDO tel que celui du modèle SIR. Parmi les plus courantes, on retrouve :

- **Méthode d'Euler explicite** : C'est la plus simple à implémenter. Elle repose sur une approximation du taux de variation à l'instant t_n pour estimer la valeur à l'instant t_{n+1}. Cependant, elle peut présenter des erreurs significatives pour des pas de temps trop grands.
- **Méthode de Runge-Kutta d'ordre 4 (RK4)** : Plus précise et plus stable, cette méthode est largement utilisée dans la simulation numérique. Elle offre un bon compromis entre précision et coût de calcul.

Exemple – Méthode d'Euler explicite En discrétisant le temps t en pas réguliers Δt, on obtient les formules d'itération suivantes :

$$\begin{cases} S_{n+1} = S_n - \Delta t \cdot \beta S_n I_n \\ I_{n+1} = I_n + \Delta t \cdot (\beta S_n I_n - \gamma I_n) \\ R_{n+1} = R_n + \Delta t \cdot \gamma I_n \end{cases}$$

avec :

- Δt : le pas de temps,
- n : l'indice de l'itération (correspondant au temps $t = n \cdot \Delta t$).

Code Python –Méthode d'Euler pour le modèle SIR

```python
def euler_sir(beta, gamma, S0, I0, R0, dt, steps):
    S, I, R = [S0], [I0], [R0]
    for _ in range(steps):
        Sn, In, Rn = S[-1], I[-1], R[-1]
        dS = -beta * Sn * In / N
        dI = beta * Sn * In / N - gamma * In
        dR = gamma * In
        S.append(Sn + dt * dS)
        I.append(In + dt * dI)
        R.append(Rn + dt * dR)
    return S, I, R
```

Figure 3.1 – Implémentation de la méthode d'Euler pour le modèle SIR

Présentation des résultats On trace l'évolution de $S(t)$, $I(t)$ et $R(t)$ sur un même graphique pour observer la dynamique épidémique.

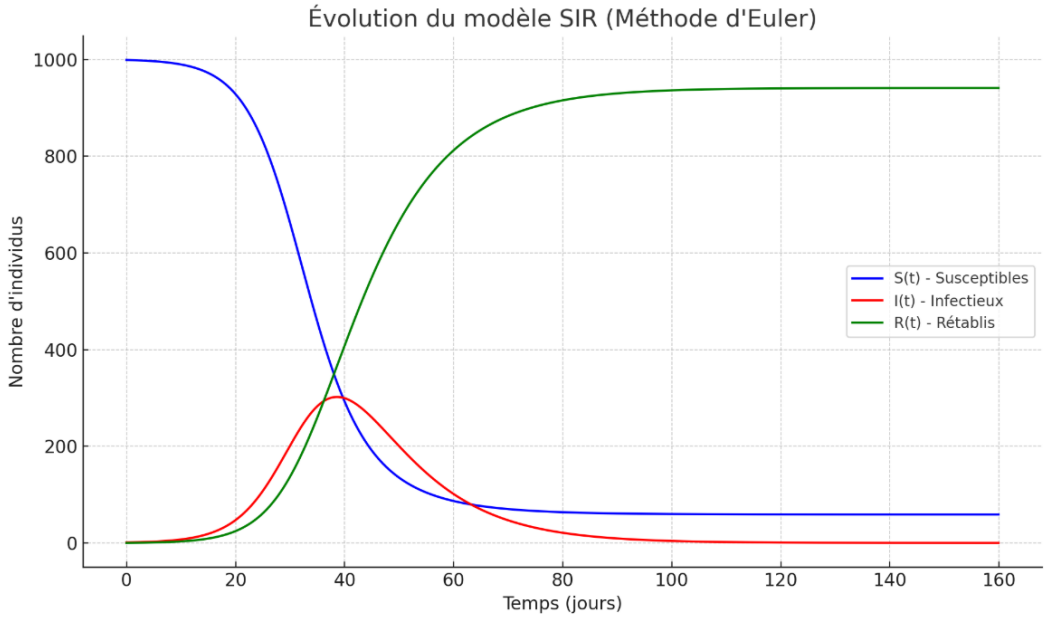

FIGURE 3.1 – Évolution du modèle SIR avec la méthode d'Euler.

Aspects pratiques d'implémentation Lorsqu'on met en œuvre une méthode numérique pour simuler le modèle SIR, certains aspects pratiques doivent être soigneusement considérés :

— **Choix du pas de temps Δt** : Un pas trop grand peut engendrer des instabilités numériques et des résultats erronés (oscillations non physiques, dépassement de valeurs logiques comme $S < 0$). À l'inverse, un pas trop petit augmente le temps de calcul. Un bon compromis doit être trouvé.

Exemple – Méthode de Runge-Kutta d'ordre 4 (RK4) La méthode de Runge-Kutta d'ordre 4 (RK4) est une méthode à pas fixe qui offre une très bonne précision sans nécessiter de calculs trop complexes. Elle est particulièrement adaptée à la résolution de systèmes d'EDO non linéaires comme celui du modèle SIR.

Pour chaque pas de temps Δt, les nouvelles valeurs de S, I et R sont obtenues à l'aide des formules suivantes :

$$\begin{cases} S_{n+1} = S_n + \frac{\Delta t}{6}(k_1^S + 2k_2^S + 2k_3^S + k_4^S) \\ I_{n+1} = I_n + \frac{\Delta t}{6}(k_1^I + 2k_2^I + 2k_3^I + k_4^I) \\ R_{n+1} = R_n + \frac{\Delta t}{6}(k_1^R + 2k_2^R + 2k_3^R + k_4^R) \end{cases}$$

avec :

$$
\begin{aligned}
k_1^S &= f_S(S_n, I_n) = -\beta S_n I_n \\
k_1^I &= f_I(S_n, I_n) = \beta S_n I_n - \gamma I_n \\
k_1^R &= f_R(I_n) = \gamma I_n
\end{aligned}
$$

$$
\begin{aligned}
k_2^S &= f_S\left(S_n + \frac{\Delta t}{2} k_1^S, I_n + \frac{\Delta t}{2} k_1^I\right) \\
k_2^I &= f_I\left(S_n + \frac{\Delta t}{2} k_1^S, I_n + \frac{\Delta t}{2} k_1^I\right) \\
k_2^R &= f_R\left(I_n + \frac{\Delta t}{2} k_1^I\right)
\end{aligned}
$$

$$
\begin{aligned}
k_3^S &= f_S\left(S_n + \frac{\Delta t}{2} k_2^S, I_n + \frac{\Delta t}{2} k_2^I\right) \\
k_3^I &= f_I\left(S_n + \frac{\Delta t}{2} k_2^S, I_n + \frac{\Delta t}{2} k_2^I\right) \\
k_3^R &= f_R\left(I_n + \frac{\Delta t}{2} k_2^I\right)
\end{aligned}
$$

$$
\begin{aligned}
k_4^S &= f_S(S_n + \Delta t k_3^S, I_n + \Delta t k_3^I) \\
k_4^I &= f_I(S_n + \Delta t k_3^S, I_n + \Delta t k_3^I) \\
k_4^R &= f_R(I_n + \Delta t k_3^I)
\end{aligned}
$$

Cette méthode utilise quatre évaluations de la fonction dérivée par pas, ce qui améliore considérablement la précision par rapport à la méthode d'Euler. Elle est donc bien adaptée pour simuler de manière stable et précise l'évolution des épidémies avec le modèle SIR.

Code Python – Méthode de Runge-Kutta d'ordre 4 (RK4)

```python
def rk4_sir(beta, gamma, S0, I0, R0, dt, steps):
    S, I, R = [S0], [I0], [R0]
    for _ in range(steps):
        Sn, In, Rn = S[-1], I[-1], R[-1]

        def dS(S, I): return -beta * S * I / N
        def dI(S, I): return beta * S * I / N - gamma * I
        def dR(I): return gamma * I

        k1_S = dt * dS(Sn, In)
        k1_I = dt * dI(Sn, In)
        k1_R = dt * dR(In)

        k2_S = dt * dS(Sn + 0.5 * k1_S, In + 0.5 * k1_I)
        k2_I = dt * dI(Sn + 0.5 * k1_S, In + 0.5 * k1_I)
        k2_R = dt * dR(In + 0.5 * k1_I)

        k3_S = dt * dS(Sn + 0.5 * k2_S, In + 0.5 * k2_I)
        k3_I = dt * dI(Sn + 0.5 * k2_S, In + 0.5 * k2_I)
        k3_R = dt * dR(In + 0.5 * k2_I)

        k4_S = dt * dS(Sn + k3_S, In + k3_I)
        k4_I = dt * dI(Sn + k3_S, In + k3_I)
        k4_R = dt * dR(In + k3_I)

        S.append(Sn + (k1_S + 2*k2_S + 2*k3_S + k4_S) / 6)
```

```
        I.append(In + (k1_I + 2*k2_I + 2*k3_I + k4_I) / 6)
        R.append(Rn + (k1_R + 2*k2_R + 2*k3_R + k4_R) / 6)
    return S, I, R
```

Figure 3.2 – Implémentation de la méthode RK4 pour le modèle SIR

Comparaison graphique Nous présentons ci-dessous la simulation du modèle SIR en utilisant la méthode de Runge-Kutta d'ordre 4 avec les mêmes paramètres que précédemment.

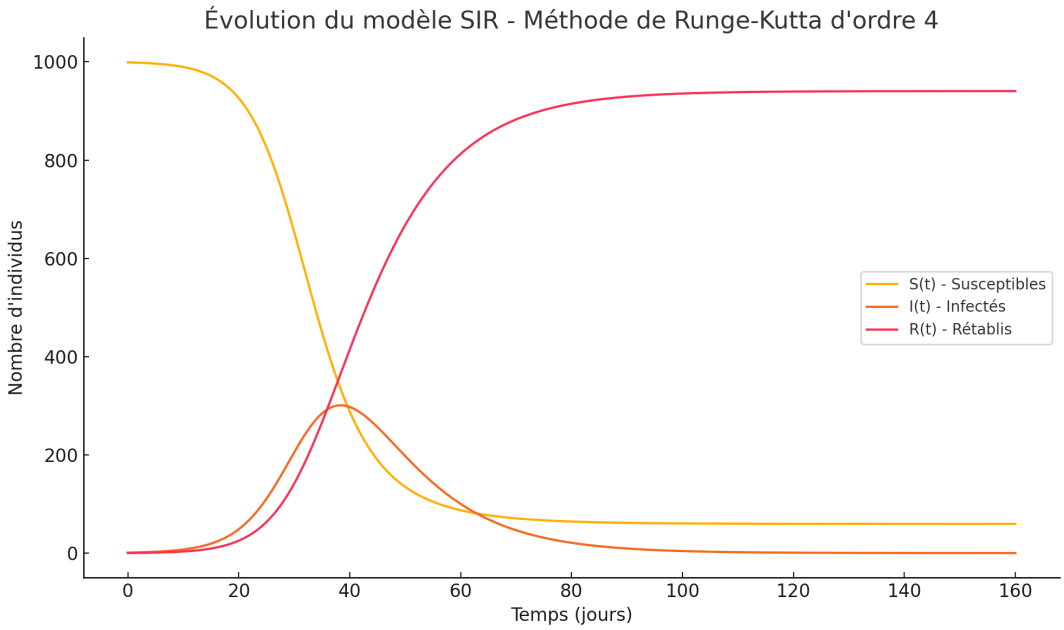

FIGURE 3.2 – Évolution du modèle SIR avec la méthode de Runge-Kutta d'ordre 4.

3.2.3 Analyse des graphiques

Comparaison entre les méthodes d'Euler et de Runge-Kutta Afin de mieux apprécier la précision des méthodes numériques, nous comparons ci-dessous l'évolution du nombre d'infectés obtenue par la méthode d'Euler et par la méthode de Runge-Kutta d'ordre 4 (RK4). Comme le montre le graphique, la méthode RK4 produit une courbe plus lisse et plus précise, notamment autour du pic épidémique, tandis que la méthode d'Euler présente des écarts notables dus à sa simplicité. Cette comparaison illustre l'intérêt d'utiliser des méthodes plus avancées comme RK4 pour simuler de manière fiable la dynamique d'une épidémie.

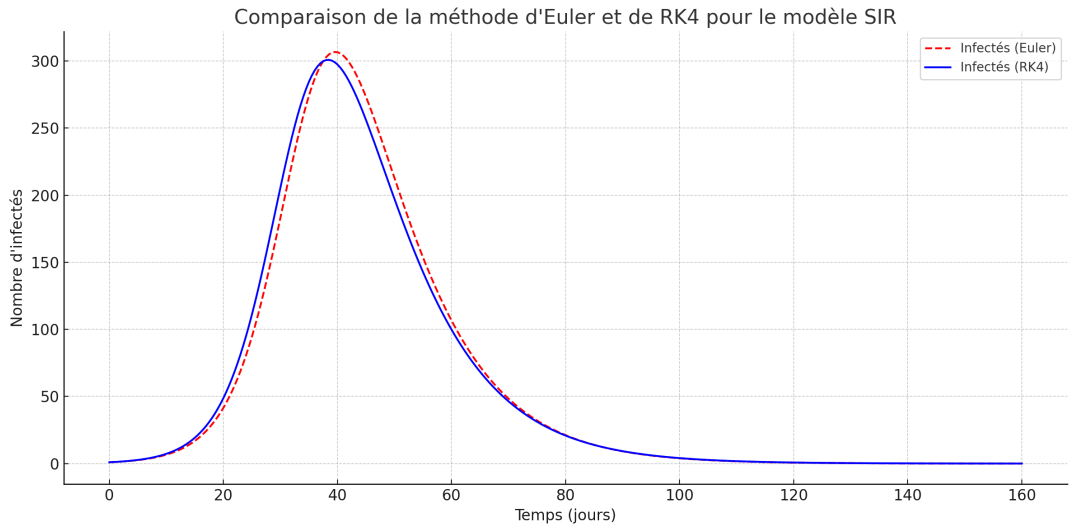

FIGURE 3.3 – Comparaison entre les résultats des méthodes d'Euler (rouge pointillé) et RK4 (bleu) pour le nombre d'infectés.

Interprétation des courbes

- $S(t)$: décroît régulièrement.
- $I(t)$: croît rapidement jusqu'au pic, puis décroît.
- $R(t)$: augmente jusqu'à atteindre un plateau.

Influence des paramètres

- Si β augmente : propagation plus rapide, pic plus haut.
- Si γ augmente : guérison plus rapide, pic plus bas.

Taux de reproduction de base

$$R_0 = \frac{\beta}{\gamma}$$

- $R_0 > 1$: épidémie se propage.
- $R_0 < 1$: l'épidémie s'éteint.

Illustration Des simulations avec des paramètres variés permettent de visualiser les effets de la vaccination (réduction de S_0) ou du confinement (réduction de β).

Limitations du modèle SIR

- Ne prend pas en compte les naissances/décès naturels.
- Immunité supposée permanente.
- Pas de distinction d'âge ou de région.
- Pas de réinfection ou mutation du virus.

3.3 Physique - Modélisation de la chute libre d'une bille

3.3.1 Introduction à la chute libre

On cherche à déterminer l'expression de la vitesse d'un corps dans l'air, c'est à dire dans notre environnement habituel. Lorsqu'on lâche un corps M de masse m dans cet environnement, il est soumis à trois forces :

— son poids : $m\vec{g}$,
— une force de frottement : $-k\vec{v}$
qui s'oppose au mouvement et qui est proportionnelle à la vitesse.
k dépend de la forme du corps et de la composition de l'atmosphère.
— la poussée d'Archimède : $\vec{\Pi}$
que l'on négligera en raison de sa faible influence.

Onoriente l'axe du déplacement vers le bas

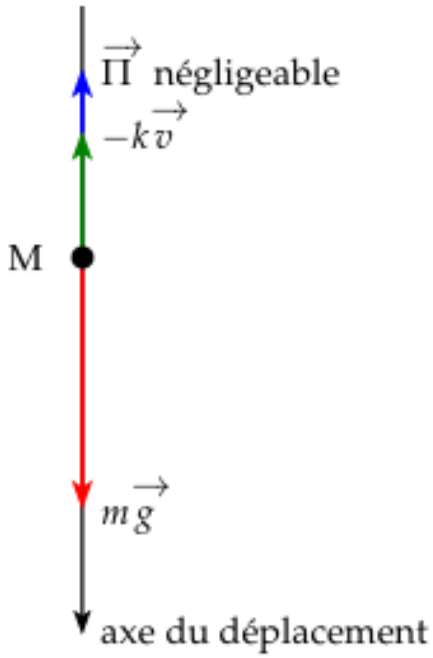

FIGURE 3.4 – La chute libre

D'après le second principe de la dynamique on a :

$$m\vec{a} = \sum \vec{F}_{\text{ext}} = m\vec{g} - k\vec{v}$$

En projetant sur l'axe du mouvement et en remarquant que l'accélération est la dérivée de la vitesse, on obtient :

$$\begin{aligned} m\frac{dv}{dt} &= mg - kv \\ \Leftrightarrow \frac{dv}{dt} &= g - \frac{k}{m}v \\ \Leftrightarrow \frac{dv}{dt} + \frac{k}{m}v &= g \quad \text{donc} \quad \tau = \frac{m}{k} \end{aligned}$$

— La solution de l'équation homogène est :

$$\lambda e^{-\frac{k}{m}t}$$

— Une solution particulière est :

$$\frac{b}{a_0} = \frac{mg}{k}$$

— La solution générale est :

$$v(t) = \lambda e^{-\frac{k}{m}t} + \frac{mg}{k}$$

Or, $v(0) = 0$ donc :

$$\boxed{v(t) = \frac{mg}{k}\left(1 - e^{-\frac{k}{m}t}\right)}$$

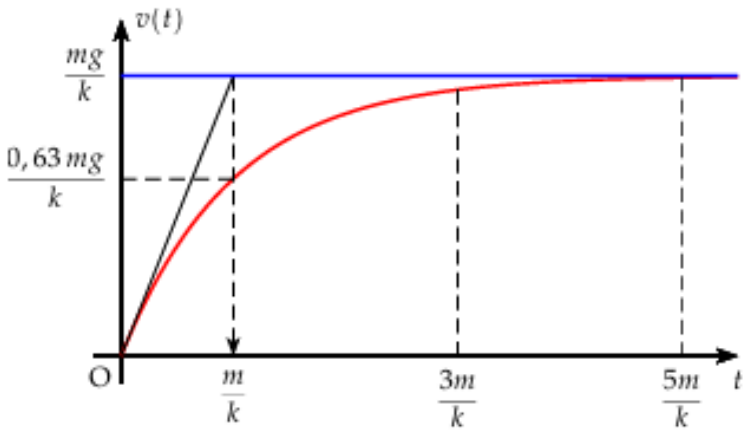

FIGURE 3.5 – I Un corps lâché en chute libre possède une vitesse limite

3.3.2 Méthode numérique de résolution

D'après la notion de dérivée, on a :

$$\frac{dv}{dt} = \lim_{\Delta t \to 0} \frac{\Delta v}{\Delta t},$$

soit en première approximation pour Δt suffisamment petit :

$$\frac{dv}{dt} \approx \frac{\Delta v}{\Delta t}.$$

Discrétisation de l'équation : En appliquant cette relation à l'équation (6), nous obtenons une suite de valeurs de vitesse v_n aux instants discrets $t_n = n\Delta t$ (pour $n = 0, 1, 2, \dots$), avec la condition initiale $v_0 = 0$.

La première itération donne :

$$v_1 - v_0 = \left[-\frac{\lambda}{m}v_0 + \left(\frac{m_{\text{air}}}{m} - 1\right)g\right]\Delta t \tag{3.1}$$

Comme $v_0 = 0$, on obtient :

$$v_1 = \left(\frac{m_{\text{air}}}{m} - 1\right)g\Delta t \tag{3.2}$$

Itérations suivantes : Les valeurs aux instants suivants (v_2, v_3, etc.) se calculent de manière analogue par la relation de récurrence :

$$v_{n+1} = v_n + \left[-\frac{\lambda}{m}v_n + \left(\frac{m_{\text{air}}}{m} - 1\right)g\right]\Delta t$$

Cette méthode numérique itérative permet de construire point par point la courbe $v(t)$ représentant l'évolution temporelle de la vitesse.

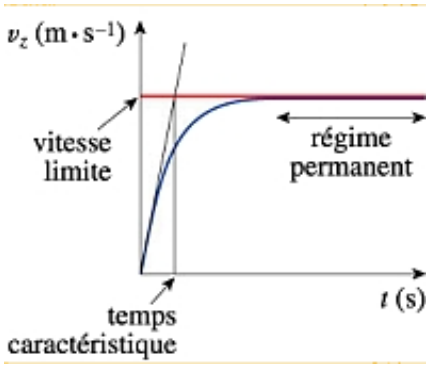

FIGURE 3.6 – Résolution de l'équation différentielle par la méthode d'Euler

3.4 Simulation graphique du modèle SIR

3.4.1 Biologie – Modèle SIR Interface de Simulation

Une interface graphique interactive a été développée pour simuler l'évolution de l'épidémie à l'aide du modèle SIR. L'utilisateur peut configurer les paramètres suivants :

- La taille de la grille (population totale),
- L'état initial de l'infection :
  - Un infecté placé au centre,
  - Un pourcentage d'individus infectés au hasard,
  - Un nombre exact d'infectés définis manuellement.

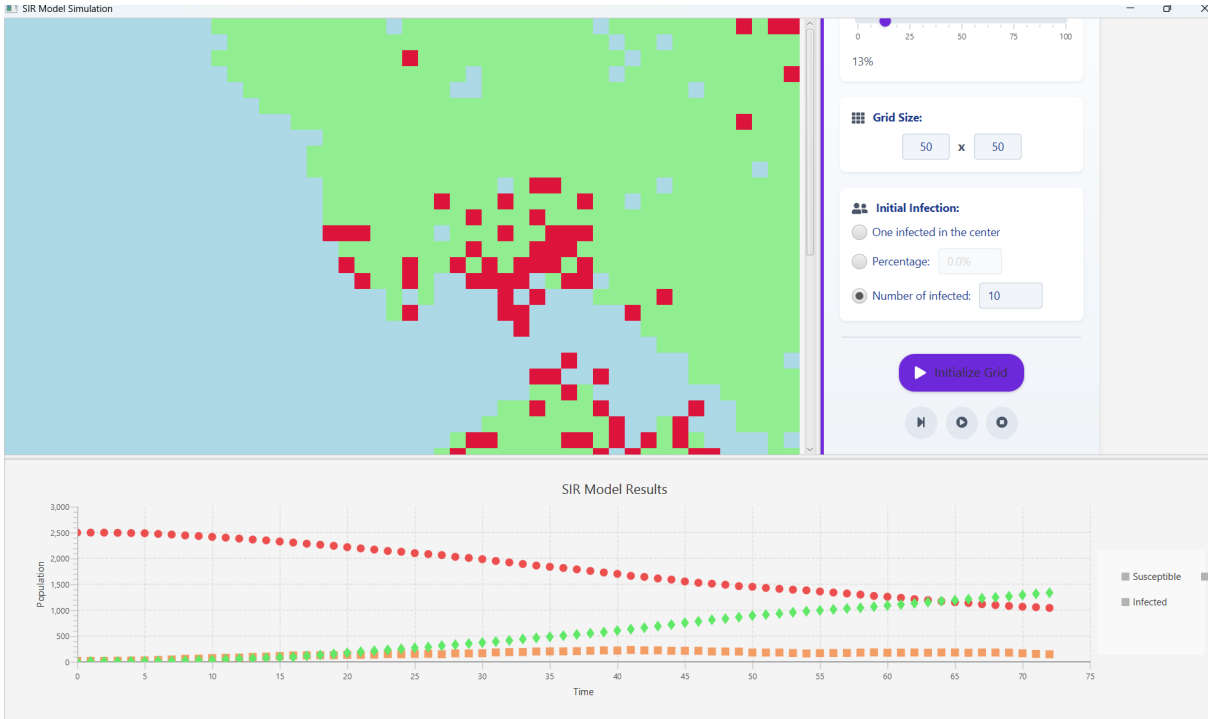

FIGURE 3.7 – Interface de simulation du modèle SIR avec grille personnalisable

Une fois la grille initialisée via le bouton `Initialize Grid`, la simulation peut être lancée. Chaque individu interagit avec ses voisins selon les règles du modèle SIR, et les transitions d'état se font au cours du temps.

Simulation spatiale du modèle SIR sur une grille de 23×50. En haut : grille représentant l'évolution de l'infection (rouge = infecté, vert = susceptible,

FIGURE 3.8 – Simulation spatiale du modèle SIR

bleu = retiré). En bas : évolution des populations Susceptibles (carrés orange), Infectées (cercles rouges) et Retirées (losanges verts) en fonction du temps.

3.4.2 Résultats de la Simulation

Un graphique dynamique est affiché en bas de l'interface pour représenter l'évolution des trois groupes de population au cours du temps :

— Susceptible : personnes non encore infectées,
— Infected : personnes actuellement infectées,
— Recovered : personnes guéries.

Cela permet d'observer :

— Le pic de l'épidémie,
— La diminution du nombre de susceptibles,
— L'augmentation des immunisés.

3.4.3 Méthodes numériques utilisées

Pour simuler le système d'équations différentielles, deux méthodes numériques ont été utilisées :

— **Méthode d'Euler** : simple à mettre en œuvre, mais approximative,
— **Méthode de Runge-Kutta d'ordre 4 (RK4)** : plus précise et stable, bien adaptée aux modèles épidémiologiques.

L'implémentation du modèle SIR via une interface interactive permet une meilleure compréhension des dynamiques épidémiques. Elle offre un outil pédagogique pour visualiser l'impact des paramètres tels que le taux de transmission (β) et le taux de guérison (γ) sur la propagation d'une maladie.

Les résultats obtenus sont cohérents avec la théorie, montrant un pic infectieux suivi d'une stabilisation lorsque la majorité de la population devient immunisée.

3.5 Simulation graphique de l'application de la chute libre

3.5.1 Interface de simulation

L'application développée permet de simuler une chute libre verticale en introduisant plusieurs paramètres initiaux : position, hauteur, vitesse initiale et pas de temps. Une interface graphique conviviale permet à l'utilisateur de configurer ces paramètres et de lancer la simulation. Plusieurs méthodes numériques sont implémentées, notamment :

— La méthode d'Euler,
— La méthode de Heun (Euler amélioré),
— La méthode de Runge-Kutta d'ordre 4 (RK4),
— Et la solution analytique de référence.

L'interface de simulation est présentée dans la figure 3.9.

FIGURE 3.9 – Interface principale de simulation

Après avoir cliqué sur « **Lancer la Simulation** », les résultats sont affichés, incluant :

— Le temps de chute pour chaque méthode,
— Une analyse des erreurs en position et en temps par rapport à la solution analytique.

3.5.2 Animation de la chute et visualisation en temps réel

L'application permet également une **visualisation animée** de la trajectoire de l'objet en chute libre, ainsi qu'une représentation graphique en temps réel de l'évolution de la position et de la vitesse.

L'exemple suivant correspond à une simulation avec une hauteur initiale de 300 m et une vitesse initiale de 5 m/s. L'objet atteint le sol après environ 8.4 secondes.

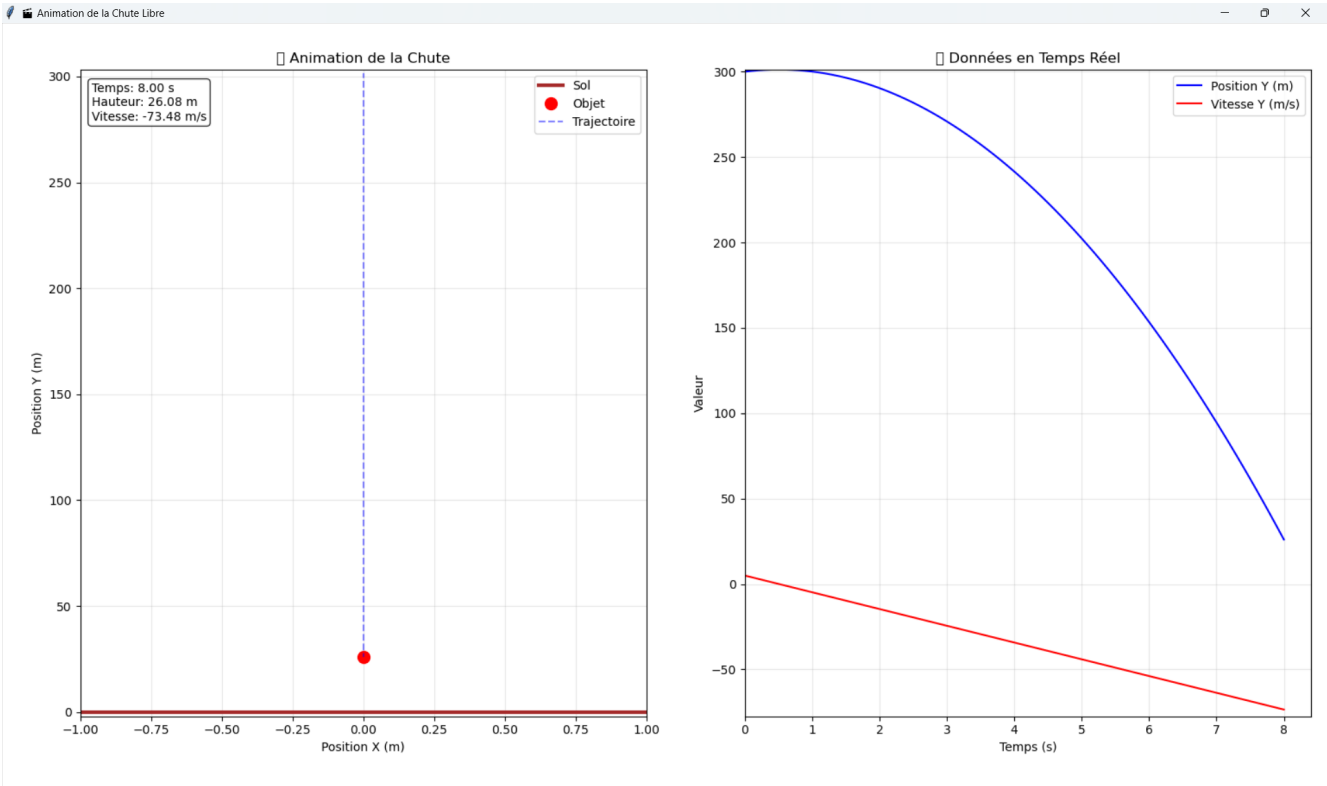

FIGURE 3.10 – Animation de la chute libre et données en temps réel

À gauche : la chute animée avec l'objet (point rouge) et sa trajectoire.
À droite : évolution de la position (bleu) et de la vitesse (rouge) en fonction du temps.

3.5.3 Analyse graphique des méthodes numériques

L'outil fournit également une comparaison graphique entre les différentes méthodes numériques utilisées. Les courbes suivantes montrent :

— L'évolution de la position Y en fonction du temps,
— L'évolution de la vitesse Y,
— L'erreur absolue par rapport à la solution analytique,
— Une comparaison du temps de chute estimé par méthode.

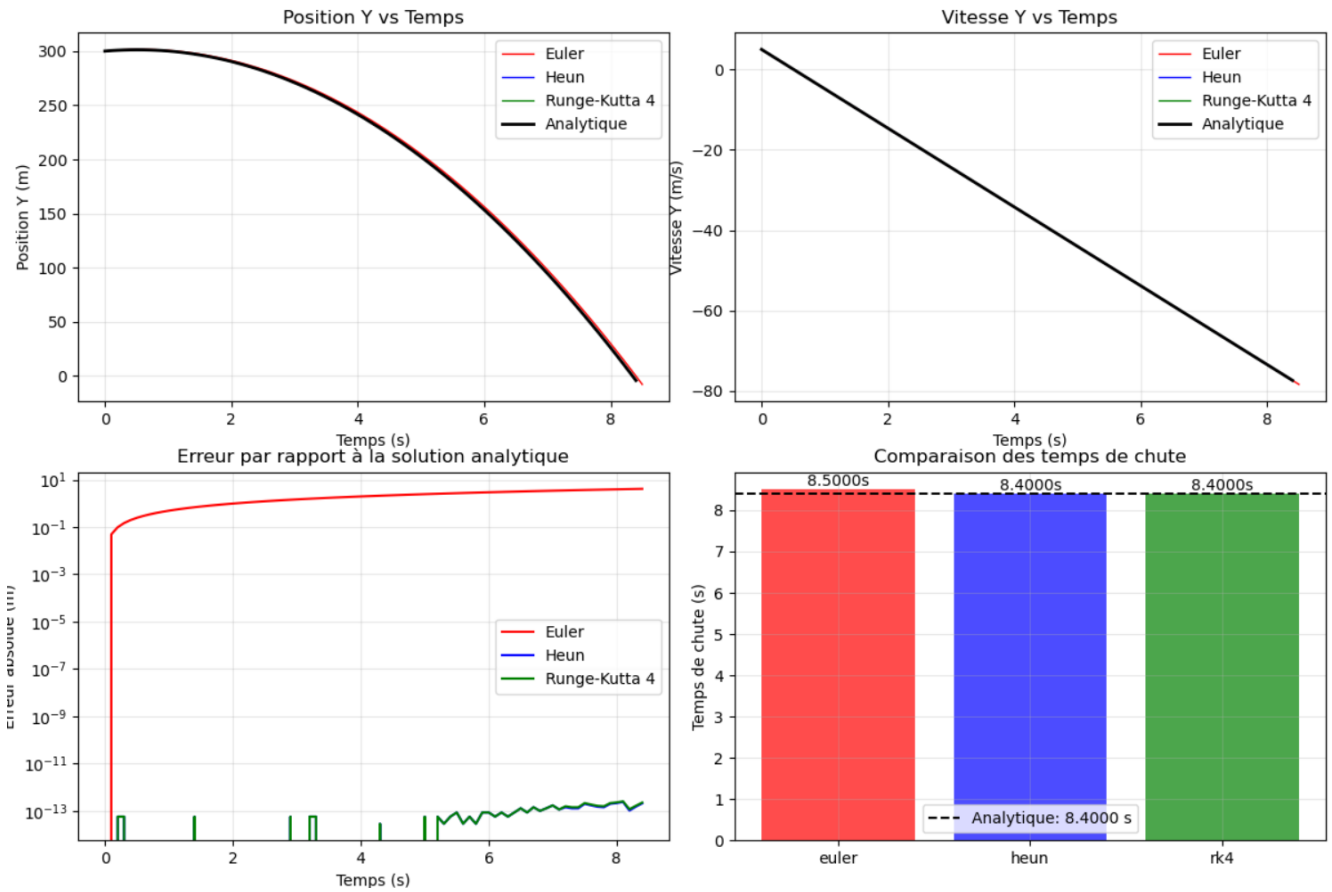

FIGURE 3.11 – Comparaison graphique des méthodes numériques

On observe que :

— La méthode d'Euler surestime légèrement le temps de chute et présente une erreur croissante dans le temps,

— Les méthodes de Heun et RK4 sont très proches de la solution analytique,

— RK4 est la plus précise, avec une erreur absolue quasi-nulle (inférieure à 10^{-13}).

Cette interface graphique permet une **visualisation claire, interactive et intuitive** du phénomène de chute libre. Elle montre également les différences de précision entre les méthodes numériques, renforçant la compréhension des modèles mathématiques et physiques derrière la simulation. Elle constitue ainsi un excellent outil pédagogique et expérimental.

3.6 Conclusion

À travers ce chapitre, nous avons mis en lumière la puissance des équations différentielles ordinaires (EDO) et des méthodes numériques dans la modélisation de phénomènes concrets et variés.

En biologie, le modèle SIR a permis de simuler la propagation d'une épidémie, en traduisant l'évolution des populations sensibles, infectées et guéries par un système d'équations différentielles. Grâce aux méthodes numériques comme Euler ou Runge-Kutta, il est possible d'analyser l'impact des paramètres du modèle (taux de transmission, taux de guérison, etc.) sur la dynamique de l'épidémie et d'explorer différents scénarios.

En physique, l'étude de la chute libre d'un objet dans l'air a illustré comment une EDO permet de modéliser le mouvement sous l'effet de forces réelles telles que le poids et les frottements. Nous avons vu que, même si une solution analytique existe dans ce cas, la méthode numérique (notamment la méthode d'Euler) permet d'obtenir une approximation efficace et progressive de la vitesse en fonction du temps.

Ces deux applications démontrent l'intérêt fondamental des EDO pour la compréhension et la prédiction de phénomènes dans des contextes très différents. Elles soulignent également l'importance des outils numériques pour surmonter les limites des solutions exactes et obtenir des résultats exploitables dans la pratique scientifique et technique.

Chapitre 4 : Implémentation numérique des méthodes avec Python

Chapitre 4

Implémentation numérique des méthodes avec Python

Introduction

L'objectif de ce chapitre est de passer de la théorie à la pratique en mettant en œuvre les méthodes numériques étudiées précédemment à l'aide du langage de programmation **Python**. Après avoir présenté les fondements mathématiques des méthodes d'Euler, de Heun et de Runge-Kutta dans les chapitres antérieurs, nous proposons ici une implémentation algorithmique et graphique de ces schémas pour la résolution d'équations différentielles ordinaires (EDO).

L'utilisation de Python se justifie par sa syntaxe simple, sa richesse en bibliothèques scientifiques telles que `NumPy` et `Matplotlib`, et sa capacité à visualiser les résultats de manière intuitive et interactive. Ce chapitre montrera ainsi comment transformer une équation différentielle théorique en un programme exécutable capable de fournir une approximation numérique de la solution.

Nous commencerons par l'implémentation des méthodes numériques pour les EDO du premier ordre, avant de traiter la transformation d'EDO d'ordre supérieur en systèmes du premier ordre. Ensuite, nous présenterons les scripts Python correspondants, accompagnés de représentations graphiques permettant de comparer les différentes méthodes en termes de précision et de stabilité.

Enfin, nous conclurons ce chapitre par une analyse comparative basée sur les résultats obtenus et les performances de chaque méthode. Cette étape mettra en évidence les avantages de la méthode de **Runge-Kutta d'ordre 4**, souvent considérée comme un compromis optimal entre précision et coût de calcul.

4.1 Méthodes numériques pour les EDO du premier ordre

Les EDO du premier ordre peuvent être résolues numériquement à l'aide de plusieurs méthodes d'approximation. Parmi les plus utilisées :

4.1.1 Méthode d'Euler

Principe de la méthode

La méthode d'Euler est une méthode numérique simple pour résoudre les équations différentielles ordinaires (EDO). Elle repose sur l'approximation suivante :

$$y(t+h) \approx y(t) + y'(t) \cdot h$$

La formule récursive de la méthode d'Euler s'écrit :

$$h = t_{n+1} - t_n$$
$$k_1 = f(t_n, y_n)$$
$$y_{n+1} = y_n + k_1 \cdot h$$

Implémentation en Python

Voici une fonction `odeEuler` en Python qui applique cette méthode :

```python
import numpy as np

def odeEuler(f, t, y0):
    y = np.zeros(len(t))
    y[0] = y0
    for n in range(len(t) - 1):
        h = t[n+1] - t[n]
        k1 = f(t[n], y[n])
        y[n+1] = y[n] + k1 * h
    return y
```

Application : $y' = y\cos(t), \quad y(0) = 1$

On souhaite résoudre :

$$y' = y\cos(t), \quad y(0) = 1$$

La solution exacte est :

$$y(t) = e^{\sin(t)}$$

Voici le code Python qui trace à la fois l'approximation d'Euler et la solution exacte :

```python
import matplotlib.pyplot as plt

f = lambda t, y: y * np.cos(t)
y0 = 1
t0 = 0
tf = 2 * np.pi
h = 0.25
N = int((tf - t0) / h) + 1
t = np.linspace(t0, tf, N)
y = odeEuler(f, t, y0)

# Trac  de la solution approch e
plt.plot(t, y, 'b.-', label='Euler')

# Trac  de la solution exacte
t_exact = np.linspace(t0, tf, 100)
y_exact = np.exp(np.sin(t_exact))
plt.plot(t_exact, y_exact, 'r', label='Exacte')

plt.grid(True)
plt.title("Approximation par la m thode d'Euler")
plt.xlabel("t")
plt.ylabel("y(t)")
plt.legend()
plt.show()
```

4.1.2 Méthode de Heun

La méthode d'Euler utilise le polynôme de Taylor de degré 1 (c'est-à-dire la ligne tangente) pour approximer $y(t)$. La **méthode de Heun** est construite à partir du polynôme de Taylor de degré 2 :

$$y(t+h) \approx y(t) + y'(t)h + \frac{y''(t)}{2}h^2$$

Nous voulons seulement utiliser $y' = f(t, y)$ dans notre approximation. Pour cela, nous introduisons la formule de différence avant pour approximer y'' en fonction de y' :

$$y''(t) \approx \frac{y'(t+h) - y'(t)}{h}$$

En combinant, on obtient :

$$y(t+h) \approx y(t) + \frac{y'(t+h) + y'(t)}{2}h$$

En utilisant la méthode d'Euler pour approximer $y'(t+h) \approx f(t+h, y(t) + y'(t)h)$, on obtient :

$$y'(t+h) \approx f(t+h, y(t) + y'(t)h)$$

La méthode de Heun est donc définie par la formule récursive en deux étapes :

$$\begin{aligned} h &= t_{n+1} - t_n \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h, y_n + k_1 h) \\ y_{n+1} &= y_n + \left(\frac{k_1 + k_2}{2}\right) h \end{aligned}$$

Écrivons une fonction `odeHeun` qui prend en entrée les paramètres f, t et y_0, où :

- `f` est une fonction représentant le second membre de l'équation $y' = f(t, y)$
- `t` est un tableau 1D de NumPy
- `y0` est la valeur initiale $y(t_0) = y_0$ où $t_0 = t[0]$

La fonction `odeHeun` retourne un tableau 1D de NumPy contenant les valeurs approchées de $y(t)$ en utilisant la méthode de Heun.

```python
def odeHeun(f, t, y0):
    y = np.zeros(len(t))
    y[0] = y0
    for n in range(0, len(t)-1):
        h = t[n+1] - t[n]
        k1 = f(t[n], y[n])
        k2 = f(t[n] + h, y[n] + k1 * h)
        y[n+1] = y[n] + (k1 + k2) / 2 * h
    return y
```

Figure 4.1 – Implémentation de la méthode de Heun

Exemple numérique

Considérons l'équation différentielle :

$$y' = y\cos(t), \quad y(0) = 1$$

dont la solution exacte est :

$$y(t) = e^{\sin(t)}$$

Traçons les solutions approchées à l'aide des méthodes d'Euler et de Heun, avec un pas $h = 0.25$, et comparons avec la solution exacte sur l'intervalle $0 \leq t \leq 2\pi$.

```python
f = lambda t, y: y * np.cos(t)
y0 = 1
t0 = 0
tf = 2 * np.pi
h = 0.25
N = int((tf - t0) / h) + 1
t = np.linspace(t0, tf, N)

y_euler = odeEuler(f, t, y0)
plt.plot(t, y_euler, 'g.-')

y_heun = odeHeun(f, t, y0)
plt.plot(t, y_heun, 'b.-')

t_exact = np.linspace(t0, tf, 50)
y_exact = np.exp(np.sin(t_exact))
plt.plot(t_exact, y_exact, 'r')

plt.grid(True)
plt.title("$y' = y \\cos(t), \\ y(0)=1$")
plt.legend(["Euler", "Heun", "Exact"])
plt.show()
```

Figure 4.2 – Comparaison entre Euler, Heun et la solution exacte

4.2.3 Méthode de Runge-Kutta d'ordre 4

La méthode de Runge-Kutta d'ordre 4 (RK4) est l'une des plus utilisées pour la résolution numérique des équations différentielles ordinaires (EDO). Elle offre un excellent compromis entre précision et coût de calcul. Contrairement à la méthode d'Euler, elle utilise quatre évaluations de la fonction dérivée à chaque pas, ce qui augmente considérablement l'exactitude.

Principe : Soit une équation différentielle du premier ordre :

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0$$

À chaque pas, la méthode calcule :

$$\begin{aligned}
k_1 &= f(x_n, y_n) \\
k_2 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\
k_3 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\
k_4 &= f(x_n + h, y_n + hk_3)
\end{aligned}$$

La solution à l'instant suivant est donnée par :

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Précision : Ordre 4 (erreur locale $\mathcal{O}(h^5)$, erreur globale $\mathcal{O}(h^4)$).

Usage : Cette méthode est recommandée pour la majorité des applications, notamment :

- Simulation de systèmes dynamiques (physique, biologie, ingénierie, etc.)
- Intégration de trajectoires (mécanique, robotique, etc.)
- Problèmes de Cauchy où la précision est importante et le temps de calcul raisonnable

Implémentation Python :

```python
def pasRK4(F, x, y, deltaX):
    k1 = deltaX * F(y, x)
    k2 = deltaX * F(y + 0.5 * k1, x + 0.5 * deltaX)
    k3 = deltaX * F(y + 0.5 * k2, x + 0.5 * deltaX)
    k4 = deltaX * F(y + k3, x + deltaX)
    return y + (k1 + 2*k2 + 2*k3 + k4) / 6.0
```

Figure 4.3 – Un pas de la méthode RK4

```python
import matplotlib.pyplot as plt

def rk4(F, t0, y0, deltaT, Tfin):
    tv = []
    yv = []
    t = t0
    y = y0
    tv.append(t)
    yv.append(y)
    while t <= Tfin + 1e-8:
        y = pasRK4(F, t, y, deltaT)
        t += deltaT
        tv.append(t)
        yv.append(y)
    return tv, yv

# Exemple  d  utilisation
def F(y, x):
    return x * y  # Exemple: dy/dx = x*y

t0 = 0
y0 = 1
deltaT = 0.1
Tfin = 5

tv, yv = rk4(F, t0, y0, deltaT, Tfin)

plt.plot(tv, yv, label="Solution RK4", color='blue')
plt.xlabel("t")
plt.ylabel("y(t)")
plt.title("R solution d'EDO par la m thode RK4")
plt.grid(True)
plt.legend()
plt.show()
```

Figure 4.4 – Intégrateur complet RK4 avec affichage graphique

4.2 Transformation des EDO d'ordre supérieur en systèmes

Les équations différentielles d'ordre supérieur peuvent être transformées en systèmes d'équations du premier ordre. Par exemple, une EDO d'ordre 2 :

$$y'' = f(x, y, y')$$

Peut être convertie en un système :

$$\begin{cases} y_1 = y \\ y_2 = y' \\ y_1' = y_2 \\ y_2' = f(x, y_1, y_2) \end{cases}$$

Cette transformation permet d'appliquer les mêmes méthodes numériques vues précédemment.

4.3 Affichage des solutions graphiques

L'affichage graphique permet de visualiser l'évolution des solutions. On peut utiliser `matplotlib` pour tracer les courbes :

```python
import numpy as np
import matplotlib.pyplot as plt

# Define the differential equation dy/dt = y - t^2 + 1
def f(t, y):
    return y - t**2 + 1

# Euler method
def euler(f, t0, y0, h, n):
    t = [t0]
    y = [y0]
    for i in range(n):
        y_new = y[-1] + h * f(t[-1], y[-1])
        t.append(t[-1] + h)
        y.append(y_new)
    return np.array(t), np.array(y)

# Heun method
def heun(f, t0, y0, h, n):
    t = [t0]
    y = [y0]
    for i in range(n):
        k1 = f(t[-1], y[-1])
        y_predict = y[-1] + h * k1
        k2 = f(t[-1] + h, y_predict)
        y_new = y[-1] + h / 2 * (k1 + k2)
        t.append(t[-1] + h)
        y.append(y_new)
    return np.array(t), np.array(y)

# Runge-Kutta 4 method
def runge_kutta(f, t0, y0, h, n):
    t = [t0]
    y = [y0]
    for i in range(n):
        k1 = f(t[-1], y[-1])
        k2 = f(t[-1] + h/2, y[-1] + h/2 * k1)
        k3 = f(t[-1] + h/2, y[-1] + h/2 * k2)
        k4 = f(t[-1] + h, y[-1] + h * k3)
        y_new = y[-1] + h / 6 * (k1 + 2*k2 + 2*k3 + k4)
        t.append(t[-1] + h)
        y.append(y_new)
```

```python
    return np.array(t), np.array(y)

# Exact solution for comparison
def exact_solution(t):
    return (t + 1)**2 - 0.5 * np.exp(t)

# Parameters
t0, y0 = 0, 0.5
h = 0.2
n = 25
t_vals = np.linspace(t0, t0 + n*h, n+1)

# Solve using each method
t_euler, y_euler = euler(f, t0, y0, h, n)
t_heun, y_heun = heun(f, t0, y0, h, n)
t_rk, y_rk = runge_kutta(f, t0, y0, h, n)
y_exact = exact_solution(t_vals)

# Plot results
plt.figure(figsize=(12, 6))
plt.plot(t_vals, y_exact, 'k-', label='Solution exacte')
plt.plot(t_euler, y_euler, 'o--', label='Euler')
plt.plot(t_heun, y_heun, 's--', label='Heun (Euler modifi )')
plt.plot(t_rk, y_rk, 'd--', label='Runge-Kutta 4')
plt.xlabel('t')
plt.ylabel('y(t)')
plt.title("Comparaison des m thodes num riques pour l'EDO dy/dt = y - t^2 + 1")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Figure 4.5 – Affichage graphique de la solution

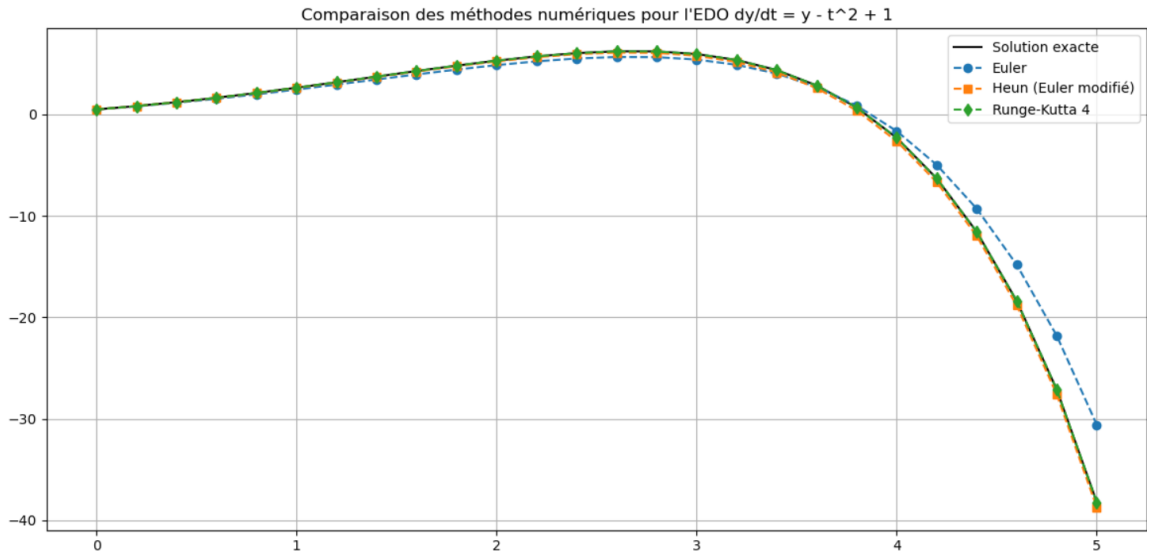

FIGURE 4.1 – Comparaison des solutions obtenues par différentes méthodes numériques avec la solution exacte.

Conclusion

Dans ce chapitre, nous avons mis en œuvre les méthodes numériques étudiées précédemment à l'aide du langage de programmation Python. Grâce à l'utilisation des bibliothèques scientifiques telles que `NumPy` et `Matplotlib`, nous avons pu coder les schémas d'Euler, de Heun et de Runge-Kutta d'ordre 4, et visualiser graphiquement les solutions approchées des équations différentielles ordinaires.

Les résultats obtenus confirment les observations théoriques : la méthode d'Euler, bien que simple, présente des erreurs importantes lorsque le pas est relativement grand ; la méthode de Heun améliore la précision de manière significative ; enfin, la méthode de Runge-Kutta d'ordre 4 offre une précision nettement supérieure, tout en restant relativement facile à implémenter.

L'implémentation en Python s'est révélée être un outil efficace non seulement pour le calcul numérique, mais aussi pour l'analyse et la visualisation des résultats. Elle constitue un support pédagogique utile pour comprendre le comportement des méthodes numériques appliquées aux équations différentielles.

Cette expérimentation pratique permet ainsi de valider les propriétés des méthodes étudiées et ouvre la voie à des extensions possibles telles que l'adaptation du pas de temps ou l'utilisation de solveurs plus avancés.

Conclusion Générale

Les équations différentielles ordinaires (EDO) constituent un outil fondamental pour la modélisation de phénomènes dynamiques dans de nombreux domaines scientifiques tels que la physique, la biologie ou encore l'ingénierie. Cependant, la recherche de solutions exactes à ces équations est souvent limitée aux cas les plus simples. Dans la majorité des situations réelles, ces solutions ne sont pas accessibles analytiquement, d'où la nécessité de recourir à des méthodes numériques.

Ce travail a permis d'explorer et de comparer plusieurs approches numériques classiques pour la résolution d'EDO, notamment la méthode d'Euler, la méthode de Heun et la méthode de Runge-Kutta d'ordre 4. Chacune de ces méthodes présente des avantages et des limites en termes de simplicité, de précision et de coût de calcul. L'analyse des erreurs globales et locales a mis en évidence l'importance du choix du pas de discrétisation pour garantir la stabilité et la convergence des solutions approchées.

L'intégration de ces méthodes dans des scripts Python a permis de visualiser concrètement les résultats à travers des simulations interactives. L'utilisation de bibliothèques telles que `NumPy` et `Matplotlib` a facilité l'implémentation algorithmique et l'interprétation graphique des résultats obtenus.

À travers des cas d'application réels, tels que le modèle SIR en épidémiologie et la modélisation de la chute libre en physique, nous avons démontré la pertinence des méthodes numériques pour résoudre des problèmes concrets, même complexes.

En conclusion, ce projet a montré que l'alliance entre mathématiques appliquées et programmation constitue un levier puissant pour l'analyse de systèmes dynamiques. La méthode de Runge-Kutta d'ordre 4 apparaît comme un compromis optimal entre efficacité, précision et simplicité de mise en œuvre, ce qui en fait un choix privilégié dans de nombreuses applications pratiques.

Bibliographie

[1] https://femto-physique.fr/analyse-numerique/runge-kutta.php.

[2] https://rtc.ma/pdfs/TSI/spe-crs/info/Les%20%C3%A9quations%20diff%C3%A9rentielles%20ordinaires.pdf#:~:text=L%E2%80%99objectif%20est%20d%E2%80%99obtenir%20une%20solution,Bien%20que%20les%20propos%20qui.

[3] https://homepages.math.uic.edu/~jan/mcs471/rungekutta.pdf.

[4] https://www.phys.uconn.edu/~rozman/Courses/m3510_17f/downloads/runge-kutta.pdf.

[5] https://www.lyceedadultes.fr/sitepedagogique/documents/math/math_pour_aller_plus_loin/en_route_vers_superieur/09_equations_differentiellles_physique.pdf.

[6] https://fr.scribd.com/embeds/31196203/content.

[7] https://www.geeksforgeeks.org/dsa/euler-method-solving-differential-equation/.

[8] https://medium.com/%40pukumarathe/eulers-method-and-runge-kutta-4th-order-method-i

[9] https://patrickwalls.github.io/mathematicalpython/differential-equations/numerical-methods/.

[10] https://www.sciencedirect.com/science/article/pii/S0025556421001565?via%3Dihub.

[11] https://mathworld.wolfram.com/Runge-KuttaMethod.html?utm_source=.

[12] https://matplotlib.org/stable/gallery/index.html.

[13] https://paulnotebook.net/home/my-code/heuns-method-in-python/?utm_source=.

[14] https://github.com/stur86/compyrtment/blob/master/cpyment/cmodel.py.

[15] https://perso.u-cergy.fr/~atorcini/MATDID/PYTHON/lect9.pdf?utm_source=.

[16] https://lemesurierb.people.charleston.edu/introduction-to-numerical-methods-and-a docs/main/ODE-IVP-1-basics-and-Euler-python.html?utm_source=.

[17] https://primer-computational-mathematics.github.io/book/c_mathematics/numerical_methods/5_Runge_Kutta_method.html.

.